

Research paper

A robust sparse identification of nonlinear dynamics approach by combining neural networks and an integral form

Ali Forootani^{ID*}, Pawan Goyal, Peter Benner

Max Planck Institute for Dynamics of Complex Technical Systems, 39106 Magdeburg, Sandtorstrasse 1, Saxony-Anhalt, Germany

ARTICLE INFO

Keywords:

Sparse regression
Discovering governing equations
Neural networks
Nonlinear system identification
Runge–Kutta scheme

ABSTRACT

One widely used methodology for uncovering governing equations from data is sparse regression for nonlinear dynamics, commonly known as Sparse Identification of Nonlinear Dynamics (SINDy). However, noisy and limited data remain a significant challenge for the success of the SINDy approach. In this work, we propose a robust strategy to discover nonlinear governing equations from both noisy and scarce data. Specifically, we employ neural networks to learn an implicit representation from measurement data, thereby ensuring that the network output remains close to the measurements while also admitting a dynamical system interpretation for its time evolution. Moreover, we identify this dynamical system in the spirit of the SINDy framework. By leveraging the neural network's implicit representation, we employ automatic differentiation to obtain the derivative information required by SINDy. To further enhance the robustness of our approach, we incorporate an integral constraint on the output of the implicit networks. In addition, we extend our method to handle data acquired from multiple initial conditions. Through several examples, we demonstrate the proposed method's effectiveness in discovering governing equations under noisy, data-scarce conditions and compare its performance against existing methods.

1. Introduction

System identification is a crucial aspect of understanding and modeling the dynamics of various physical, chemical, and biological systems. Over the years, various powerful and efficient system identification techniques have been developed, and these methods have been applied in a wide range of applications, see, e.g., Ljung (1999), Van Overschee and de Moor (1996) and Tangirala (2018). Traditionally, system identification techniques rely on prior model hypotheses. With a linear model hypothesis, several methodologies have been proposed; see, e.g., Ljung (1999) and Van Overschee and de Moor (1996). However, for nonlinear system identification, defining a prior is challenging, and it is often done with the help of practitioners. Despite several earlier works Kumpati and Kannan (1990), Suykens et al. (1996) and Crutchfield and McNamara (1987), nonlinear system identification is still an active and exciting research field. Towards automatic nonlinear system identification, generic algorithms and symbolic regression have shown their effectiveness and promises in discovering governing nonlinear equations using measurement (Schmidt et al., 2011; Schmidt and Lipson, 2009). However, their computational expenses remain undesirable.

Instead of building suitable functions in the spirit of symbolic regression, there has been a focus on sparsity-promoting approaches for nonlinear system identification (Wang et al., 2011; Schaeffer et al., 2013; Brunton et al., 2016a). They rely on the assumption that nonlinear dynamics can be defined by a few nonlinear basis functions from a dictionary with a large collection of nonlinear basis functions. Such a technique enables the discovery of interpretable, parsimonious, and generalizable models that balance precision and performance. It is nowadays widely referred to as SINDy (Brunton et al., 2016a).

Sparse modeling of physical dynamical systems, as in SINDy approach, is crucial not only for simplifying complex models but also for discovering governing equations directly from data, especially when no first-principles model is available. This is particularly valuable in fields like biology, neuroscience, and epidemiology, where the underlying dynamics are complex and not well understood. By identifying the most influential terms, sparse modeling enhances interpretability, facilitates analysis, and improves generalization by focusing on core dynamics while reducing sensitivity to noise or irrelevant features. Additionally, it lowers computational complexity, enabling efficient simulation and real-time applications even in data-limited or noisy scenarios (Machado and Jones, 2024).

* Corresponding author.

E-mail addresses: forootani@mpi-magdeburg.mpg.de, aliforootani@ieee.org (A. Forootani), goyalp@mpi-magdeburg.mpg.de (P. Goyal), benner@mpi-magdeburg.mpg.de (P. Benner).

<https://doi.org/10.1016/j.engappai.2025.110360>

Received 17 October 2024; Received in revised form 6 February 2025; Accepted 18 February 2025

Available online 14 March 2025

0952-1976/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

SINDy has been employed for a number of challenging model discovery problems such as fluid dynamics (Loiseau and Brunton, 2018), plasma dynamics (Dam et al., 2017), turbulence closures (Beetham and Capececiatratro, 2020), mesoscale ocean closures (Zanna and Bolton, 2020), nonlinear optics (Sorokina et al., 2016), computational chemistry (Boninsegna et al., 2018), and numerical integration (Thaler et al., 2019). Moreover, the results of SINDy have been extended widely to, such as nonlinear model predictive control (Kaiser et al., 2018), rational functions (Kaheman et al., 2020; Goyal and Benner, 2022), enforcing known conservation laws and symmetries (Loiseau and Brunton, 2018), promoting stability (Kaptanoglu et al., 2021), generalizations for stochastic dynamics (Callahan et al., 2021), Bayesian perspective (Zhang and Lin, 2018).

Often, SINDy approaches require a reliable estimate of the derivative information, making them very challenging for noisy and scarce data regimes. Blending numerical methods (Schaeffer and McCalla, 2017; Kang et al., 2021; Keller and Du, 2021; Goyal and Benner, 2022; Messenger and Bortz, 2021) and weak formulations of differential equations (Messenger and Bortz, 2021) avoid these requirements, but their performance still deteriorates for low signal-to-noise measurements. In addition, the method in Messenger and Bortz (2021) relies on the choice of basis functions that allow to write differential equations in a weak formulation. The work in Fasel et al. (2022) utilizes the concepts of an ensemble to improve the predictions, but it still requires reliable estimates of derivatives to some extent. To discover governing equations from noisy data, the authors in Kaheman et al. (2022) proposed a scheme that aims to decompose the noisy signals into clean signals and the noise using a Runge–Kutta-based integration method. However, the scheme explicitly estimates the noise, making it harder to scale, and requires all the dependent variables to be available at the same time grid.

Advancements in scientific machine learning, such as Reinforcement Learning (RL), have focused on combining efficient model-based techniques with interpretable frameworks for dynamic systems (Rackauckas et al., 2020). For instance, kernel-based methods have been applied in sensor scheduling (Forootani et al., 2024a), least-squares Monte Carlo simulation methods have been used for optimal stopping time and machine replacement problems (Forootani et al., 2021, 2023), and a finite time horizon reinforcement learning approach with unweighted sample-based least-squares has been utilized in resource allocation (Forootani et al., 2019), demonstrating the effectiveness of these techniques in decision-making and optimization problems. Building on these advancements, methods like SINDy-RL (Zolman et al., 2024) combine the SINDy framework with RL to develop interpretable, efficient, model-based RL algorithms. SINDy-RL leverages sparse models for dynamics learning, facilitating policy optimization and control in low-data regimes, and represents a step toward integrating interpretable system identification with reinforcement learning paradigms.

Recently deep neural networks (DNN), have received attention in sparse regression model discovery methods. For instance, in Both et al. (2021), a deep learning-based discovery algorithm has been employed to identify underlying (partial) differential equations. However, therein, only a single trajectory is considered to recover governing equations, but in many complex processes, we might require data for different parameters and initial conditions to explore rich dynamics, thus, the reliable discovery of governing equations. Furthermore, the work Both et al. (2021) discovers governing equations based on estimating derivative information using automatic differential tools. However, differential equations can also be written in integral form, whereby numerical approaches can also be incorporated, see, e.g., Goyal and Benner (2022), Chen et al. (2018).

In this paper, we discuss an approach, named iNeural-SINDy, for the data-driven discovery of nonlinear dynamical systems using noisy data through the lens of SINDy. For this, we make use of DNN to learn an implicit representation based on the given data set so

that the network outputs denoised data, which is later utilized for the sparse regression to discover governing equations. To solve the sparse regression problem, we make use of not only automatic differentiation tools but also integral forms differential equations. As a result, we observe a robust discovery of governing equations. We note that such a concept has recently been used in the context of neural ODEs in Goyal and Benner (2023) to learn black-box dynamics using noisy and scarce data. We further discuss how to incorporate the data coming from multiple initial conditions.

It is worth to highlight that the basic SINDy algorithm, while effective for discovering governing equations from clean data, suffers from several limitations, including sensitivity to noise, reliance on numerical differentiation (which amplifies errors in noisy data), and poor performance with sparse or irregularly sampled data. Additionally, it lacks mechanisms to leverage integral forms of the dynamics, which could enhance robustness. iNeural-SINDy addresses these limitations by using neural networks to learn a smooth, implicit representation of the data, enabling accurate derivative estimation via automatic differentiation without relying on numerical methods. It further incorporates integral constraints (e.g., through Runge–Kutta integration schemes), which improves its resilience to noise and data scarcity. Moreover, iNeural-SINDy effectively handles data collected from multiple trajectories, a feature absent in the basic SINDy framework, and significantly outperforms it in noisy and complex scenarios, making it a robust tool for real-world applications.

The rest of this paper is organized as follows. Section 2 briefly recalls the SINDy approach (Brunton et al., 2016a). In Section 3, we propose a novel methodology for sparse regression to learn underlying governing equations by making use of DNN, automatic differential tools, and numerical methods. Furthermore, in Section 4, we discuss its extension for multiple initial conditions and different parameters. In Section 5, we demonstrate the proposed framework by means of various synthetic noisy measurements and present a comparison with the current state-of-the-art approaches. Finally, Section 6 concludes the paper with a brief summary and future research avenues.

2. A brief recap of SINDy

The SINDy algorithm identifies sparse dynamical system models directly from data by fitting derivatives of state variables to a library of candidate functions using sparse regression, thus deriving interpretable equations that govern the system's behavior (Brunton et al., 2016a). The SINDy algorithm is a nonlinear system identification approach which is based on the hypothesis that governing equations of a nonlinear system can be given by selecting a few suitable basis functions, see, e.g., Brunton et al. (2016a). Precisely, it aims at identifying a few basis functions from a dictionary, containing a large number of candidate basis functions. In this regard, sparsity-promoting approaches can be employed to discover parsimonious nonlinear dynamical systems to have a good trade-off for model complexity and accuracy (Brunton et al., 2014; Mackey et al., 2014). SINDy facilitates the extraction of the most salient dynamical features from a plethora of potential influences, thereby enhancing the interpretability and analytical tractability of the model (Brunton et al., 2016b). Furthermore, by concentrating on essential interactions, sparse models inherently mitigate the risk of overfitting to noisy data or idiosyncratic details not representative of the underlying physical processes (Rudy et al., 2017). This results in models with improved predictive accuracy and generalizability across different regimes and conditions (Peterson et al., 2024; Forootani and Benner, 2024; Kutz et al., 2016). Moreover, the reduced computational complexity associated with sparse representations enables more efficient numerical simulations and analyses, crucial for managing large-scale systems or real-time processing scenarios (Fasel et al., 2022).

Consider the problem of discovering nonlinear systems of the form:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)), \quad (1)$$

where $\mathbf{x}(t) = [\mathbf{x}_1(t), \mathbf{x}_2(t), \dots, \mathbf{x}_n(t)]^\top \in \mathbb{R}^n$ denotes the state at time t , and $\mathbf{f}(\mathbf{x}(t)) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a nonlinear function of the state $\mathbf{x}(t)$.

Towards discovering the function $\mathbf{f}$ in (1), which defines the vector field or dynamics of the underlying system, we start by collecting time-series data of state $\mathbf{x}(t)$. Let us further assume to have time derivative information of the state. If it is not readily available, we can approximate it using numerical methods, e.g., a finite difference scheme. Thus, consider that the data $\{\mathbf{x}(t_0), \dots, \mathbf{x}(t_N)\}$ and its derivative $\{\dot{\mathbf{x}}(t_0), \dots, \dot{\mathbf{x}}(t_N)\}$ are given. In the next step, we assemble the data in matrices as follows:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}(t_1)^\top \\ \mathbf{x}(t_2)^\top \\ \vdots \\ \mathbf{x}(t_N)^\top \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1(t_1) & \mathbf{x}_2(t_1) & \dots & \mathbf{x}_n(t_1) \\ \mathbf{x}_1(t_2) & \mathbf{x}_2(t_2) & \dots & \mathbf{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_1(t_N) & \mathbf{x}_2(t_N) & \dots & \mathbf{x}_n(t_N) \end{bmatrix}, \quad (2)$$

where each row represents a snapshot of the state. Similarly, we can write the time derivative as follows:

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}(t_1)^\top \\ \dot{\mathbf{x}}(t_2)^\top \\ \vdots \\ \dot{\mathbf{x}}(t_N)^\top \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{x}}_1(t_1) & \dot{\mathbf{x}}_2(t_1) & \dots & \dot{\mathbf{x}}_n(t_1) \\ \dot{\mathbf{x}}_1(t_2) & \dot{\mathbf{x}}_2(t_2) & \dots & \dot{\mathbf{x}}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{\mathbf{x}}_1(t_N) & \dot{\mathbf{x}}_2(t_N) & \dots & \dot{\mathbf{x}}_n(t_N) \end{bmatrix}. \quad (3)$$

The next key building block in the SINDy algorithm is the construction of a dictionary $\Theta(\mathbf{X})$, containing candidate basis functions (e.g., constant, polynomial or trigonometric functions). For instance, our dictionary matrix can be given as follows:

$$\Theta(\mathbf{X}) = \begin{bmatrix} 1 & \mathbf{X} & \mathbf{X}^2 & \mathbf{X}^3 & \dots & \sin(\mathbf{X}) & \cos(\mathbf{X}) & \sin(2\mathbf{X}) & \cos(2\mathbf{X}) & \dots \end{bmatrix}, \quad (4)$$

assume $\Theta(\mathbf{X}) \in \mathbb{R}^{n \times D}$ above formulations polynomial terms are denoted by $\mathbf{X}^2$ or $\mathbf{X}^3$; to be more descriptive $\mathbf{X}^2$ denotes the quadratic nonlinearities of the state $\mathbf{X}$ as follows:

$$\mathbf{X}^2 = \begin{bmatrix} \mathbf{x}_1^2(t_1) & \mathbf{x}_1(t_1)\mathbf{x}_2(t_1) & \dots & \mathbf{x}_2^2(t_1) & \mathbf{x}_2(t_1)\mathbf{x}_3(t_1) & \dots & \mathbf{x}_n^2(t_1) \\ \mathbf{x}_1^2(t_2) & \mathbf{x}_1(t_2)\mathbf{x}_2(t_2) & \dots & \mathbf{x}_2^2(t_2) & \mathbf{x}_2(t_2)\mathbf{x}_3(t_2) & \dots & \mathbf{x}_n^2(t_2) \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_1^2(t_N) & \mathbf{x}_1(t_N)\mathbf{x}_2(t_N) & \dots & \mathbf{x}_2^2(t_N) & \mathbf{x}_2(t_N)\mathbf{x}_3(t_N) & \dots & \mathbf{x}_n^2(t_N) \end{bmatrix}.$$

In this setting, each column of the dictionary $\Theta(\mathbf{X})$ denotes a candidate function in defining the function $\mathbf{f}(\mathbf{x})$ in (1). We are interested in identifying a few candidate functions from the dictionary Θ so that a weighted sum of these selected functions can describe the function $\mathbf{f}$. For this, we can set up a sparse regression formulation to achieve this goal. Precisely, we seek to identify a sparse vector $\Xi = [\xi_1, \xi_2, \dots, \xi_n]$, where $\xi_i^\top \in \mathbb{R}^D$ with D denoting the number of columns in Θ , that determines which features from the dictionary are active and their corresponding coefficients.

The SINDy algorithm formulates the sparse regression problem as an optimization problem as follows. Given a set of observed data $\mathbf{X}$ and the corresponding time derivatives $\dot{\mathbf{X}}$, the goal is to find the sparsest matrix Ξ that fulfills the following:

$$\dot{\mathbf{X}} = \Theta(\mathbf{X})\Xi.$$

However, finding such a matrix is an NP hard problem. Therefore, there is a need to a sparsity promoting regularization, and in this category, LASSO is a widely known approach (Hastie et al., 2009; Tibshirani, 1996).

Despite its success the LASSO algorithm (Hastie et al., 2009; Tibshirani, 1996) requires a prior information about how many non-zeros elements are expected in the matrix Ξ (Beck and Eldar, 2013; Yang

Algorithm 1 SINDy algorithm (Fasel et al., 2022)

Input: Dictionary Θ , time-series data $\mathbf{X}$, time derivative information $\dot{\mathbf{X}}$, threshold value tol , and maximum iterations max-iter .

Output: Estimated coefficients Ξ that define governing equations for nonlinear systems.

```

1:  $\Xi = (\Theta^\top \Theta) \backslash \Theta^\top \dot{\mathbf{X}}$     ▷ For initial guess, solving a least-squares problem
2:  $k = 1$ 
3: while  $k < \text{max-iter}$  do
4:    $\text{small\_inds} = (\text{abs}(\Xi) < \text{tol})$     ▷ identifying small coefficients
5:    $\Xi(\text{small\_inds}) = 0$                 ▷ excluding small coefficients
6:   Solve  $\Xi = (\Theta^\top \Theta) \backslash \Theta^\top \dot{\mathbf{X}}$  subject to  $\Xi(\text{small\_inds}) = 0$ 
7:    $k = k + 1$ 

```

et al., 2016), which is not known. On the other hand, the authors in Brunton et al. (2016a) discuss a sequential thresholding approach, where simple least squares problems are solved iteratively, and at each step, coefficients below a given tolerance are pruned. Analysis of algorithm is discussed in Zhang and Schaeffer (2019). We summarize the SINDy approach in Algorithm 1. Moreover, we mention that other regularization schemes or heuristics are discussed in Rudy et al. (2017), Schaeffer and McCalla (2017), Kaptanoglu et al. (2021), Reinbold et al. (2020) and Goyal and Benner (2022), but in this work, we focus only on the sequential thresholding approach, similar to in Algorithm 1 due to its simplicity.

3. iNeural-SINDy: Neural networks and integrating schemes assisted SINDy approach

A challenge in the classical SINDy approach discussed in the previous section is the availability of an accurate estimate of the derivative information. If the derivative information is inaccurate, the resulting sparse model may not accurately capture the underlying system dynamics.

In this section, we present an approach that combines SINDy framework with a numerical integration scheme and neural networks in a particular way so that a robust discovery of governing equations can be made amid poor signal-to-noise ratio and irregularities in data. The methodology is inspired by the work Goyal and Benner (2023). The main components of the methodology are as follows. For given noisy data, we aim to learn an implicit representation using a neural network based on the noisy data so that the network yields denoised data in the vicinity of the noisy collected data, and governing equations describing the dynamics of the denoised data can be obtained by employing SINDy. For SINDy, we utilize automatic differential tools to obtain the derivative information via the network and also make use of an integral form of the differential equations. In the following, we make these discussions more precise.

Consider noisy data $\mathbf{y}(t) \in \mathbb{R}^n$ at the time instances $\{t_0, \dots, t_N\}$, i.e., $\{\mathbf{y}(t_0), \dots, \mathbf{y}(t_N)\}$. Moreover, $\mathbf{y}(t) = \mathbf{x}(t) + \epsilon(t)$, where $\mathbf{x}(t)$ and $\epsilon(t)$ denote clean data and noise, respectively. Under this setting, we aim to discover the structure of vector field $\mathbf{f}$ by identifying the most active terms in the dictionary Θ so that it satisfies

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}). \quad (5)$$

Note that we do not know ϵ . In order to learn $\mathbf{f}$ from $\mathbf{y}$, we blend three ingredients, which are discussed in the following.

- (a) **Sparse regression assumption:** In our setting, we utilize the principle of SINDy, which we discussed in the previous section. This means that the system dynamics (or vector field defining dynamics) can be represented by a few suitable terms from a dictionary of candidate functions. This allows us to obtain

a parsimonious representation of dynamical systems and reduces the model complexity, leading to better generalization and interpretability of the models.

- (b) **Automatic differentiation to estimate derivative information:** As mentioned earlier, SINDy requires accurate derivative information for the system, which can be challenging to obtain from experiments or to estimate using numerical methods. To cope with this issue, we make use of neural networks with its automatic differentiation (AD) feature, which is a technique used to estimate derivative information. The use of a DNN in combination with the SINDy algorithm was earlier discussed in [Both et al. \(2021\)](#), where the discovery of nonlinear system dynamics without explicit need of accurate derivative information ([Both et al., 2021](#)).

We make use of DNN to parameterize nonlinear mapping from time t to the dependent variable $y(t)$. To that end, let us denote a DNN by $\mathcal{G}_\theta$, where θ contains DNN parameters. The input to $\mathcal{G}_\theta$ is time t , and its output is $y(t)$, i.e., $y(t) = \mathcal{G}_\theta(t)$. However, in the case of noisy measurement $y(t)$ at time $\{t_0, \dots, t_{\mathcal{N}}\}$, we expect $\mathcal{G}_\theta$ to predict outputs in the proximity of y , i.e.,

$$y(t) \approx \mathbf{x}(t) = \mathcal{G}_\theta(t), \quad t = \{t_0, \dots, t_{\mathcal{N}}\}.$$

With the sparse regression hypothesis, we aim to learn a dynamical model for $\mathbf{x}$, as it can be seen as a denoised version of y . For this, we construct a dictionary of possible candidate functions using $\mathbf{x}$, which we denote by $\Theta(\mathbf{X}(t))$. Next, we require the derivative information of $\mathbf{x}$ with respect to time t . Since we have an implicit representation of $\mathbf{x}(t)$ using a DNN, we can employ AD to obtain the required information. Having dictionary and derivative information, we set up a sparse regression problem as follows:

$$\dot{\mathbf{X}}(t) = \Theta(\mathbf{X}(t))\Xi, \quad (6)$$

where Ξ is the sparsest possible matrix, which selects the most active terms from the dictionary to define dynamics. Finding the sparsest solution is computationally infeasible; we, thus, utilize the sequential thresholding approach as discussed in Algorithm 1 with minor modifications. Instead of solving least-squares problems in Steps 1 and 5 in Algorithm 1, we consider a loss function as follows:

$$\mathcal{L} := \min_{\theta, \Xi} \sum_{i=0}^{\mathcal{N}} \lambda_1 \|y(t_i) - \mathbf{x}(t_i)\| + \lambda_2 \|\dot{\mathbf{x}}(t) - \Theta(\mathbf{x}(t))\Xi\|, \quad (7)$$

where $\mathbf{x}(t_i) := \mathcal{G}_\theta(t_i)$, and λ_1 and λ_2 are hyperparameters.

- (c) **Numerical integration scheme:** A dynamical system is a particle or an ensemble of particles whose state varies over time and thus obeys differential equations involving time derivatives ([Sprott and Sprott, 2003](#)). To predict the evolution of the dynamical system, it is necessary to have an analytical solution of such equations or their integration over time through computer simulations. Therefore, we aim to incorporate the information contained in the form of integration of dynamical systems while discovering governing equations via sparse regression, which can make the process of discovering equations robust to the noise and scarcity of data due to the additional information about integration.

When differential equations are written in an integral form, then we do not require derivative information as well; however, the resulting optimization problem involves an integral. In this regard, one can employ the principle of Neural-ODEs ([Chen et al., 2018](#)) to solve efficiently such optimization problems. One can also approximate the integral form using suitable integrating schemes ([Butcher, 2016](#)), and recently, fourth-order Runge–Kutta (RK4) scheme ([Goyal and Benner, 2022](#)) and linear multi-step methods ([Buckwar and Winkler, 2006](#)) are combined

with SINDy. In this work, we make use of the RK4 scheme to approximate an integral.

Following [Goyal and Benner \(2022\)](#), our goal is to predict the state of a dynamic system $\mathbf{x}(t_{k+1})$ at time $t = t_{k+1}$ from the state $\mathbf{x}(t_k)$ at time $t = t_k$, where $k \in \{0, 1, \dots, \mathcal{N} - 1\}$. By employing the RK4 scheme, $\mathbf{x}(t_{k+1})$ can be computed as a weighted sum of four components that are the product of the time-step and gradient field information $\mathbf{f}(\cdot)$ at the specific locations. These components are computed as follows:

$$\mathbf{x}(t_{k+1}) \approx \mathbf{x}(t_k) + \frac{1}{6} h_k (\mathbf{a}_1 + 2 \cdot \mathbf{a}_2 + 2 \cdot \mathbf{a}_3 + \mathbf{a}_4), \quad h_k = t_{k+1} - t_k, \quad (8)$$

where,

$$\begin{aligned} \mathbf{a}_1 &= \mathbf{f}(\mathbf{x}(t_k)), \quad \mathbf{a}_2 = \mathbf{f}\left(\mathbf{x}(t_k) + h_k \frac{\mathbf{a}_1}{2}\right), \quad \mathbf{a}_3 = \mathbf{f}\left(\mathbf{x}(t_k) + h_k \frac{\mathbf{a}_2}{2}\right), \\ \mathbf{a}_4 &= \mathbf{f}\left(\mathbf{x}(t_k) + h_k \mathbf{a}_3\right). \end{aligned}$$

For the sake of simplicity with a slight abuse of a notation, the right-hand side of (8) is denoted by $\mathcal{F}_{\text{RK4}}(\mathbf{f}, \mathbf{x}(t_k), h_k)$, i.e.,

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k + h_k) \approx \mathcal{F}_{\text{RK4}}(\mathbf{f}, \mathbf{x}(t_k), h_k). \quad (9)$$

Like the SINDy algorithm, we collect samples from the dynamical system at time $t = \{t_0, \dots, t_{\mathcal{N}}\}$ and define the time step as $h_k := t_{k+1} - t_k$.

With sparse regression assumption, we can write $\mathbf{f}(\mathbf{x}) = \Theta(\mathbf{x})\Xi$, where $\Theta(\mathbf{x})$ is a dictionary and Ξ is a sparse matrix. Then, we can set up a sparse regression as follows. We seek to identify the sparsest matrix Ξ so that the following is minimized:

$$\sum_k \|\mathbf{x}(t_{k+1}) - \mathcal{F}_{\text{RK4}}(\Theta(\mathbf{x})\Xi, \mathbf{x}(t_k), h_k)\|.$$

When the RK4 scheme is merged with the previously discussed DNN framework, we apply a one-time ahead prediction based on RK4–SINDy to the output of our DNN, i.e.,

$$\mathbf{x}_{\text{RK4}}(t_{k+1}) \approx \mathcal{F}_{\text{RK4}}(\mathbf{f}, \mathbf{x}(t_k), h_k).$$

Having all these ingredients, we combine them to define a loss function to train our DNN structure, as well as to discover governing equations describing underlying dynamics. To that end, we have the following loss function:

$$\mathcal{L} = \mu_1 \mathcal{L}_{\text{MSE}} + \mu_2 \mathcal{L}_{\text{deri}} + \mu_3 \mathcal{L}_{\text{RK4}}, \quad \mu_1, \mu_2, \mu_3 \in [0, 1], \quad (10)$$

where $\mathcal{L}_{\text{MSE}}$ is the mean square error (MSE) of the output of the DNN $\mathcal{G}_\theta$ (denoted by $\hat{\mathbf{x}}$) with respect to the collected data y , and $\{\mu_1, \mu_2, \mu_3\}$ are positive constants, determining the weight of different losses in the total loss function.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{\mathcal{N}} \sum_{k=1}^{\mathcal{N}} \left\| y(t_k) - \mathbf{x}(t_k) \right\|_2^2 \quad (11)$$

forces the DNN to produce output in the vicinity of the measurements, and μ_1 is its weight. $\mathcal{L}_{\text{deri}}$ is inspired by the sparse regression and aims to compute the sparse coefficient matrix Ξ . It is as follows:

$$\mathcal{L}_{\text{deri}} = \frac{1}{\mathcal{N}} \sum_{k=1}^{\mathcal{N}} \left\| \dot{\mathbf{x}}(t_k) - \Theta(\mathbf{x}(t_k))\Xi \right\|_2^2, \quad (12)$$

The term $\mathcal{L}_{\text{RK4}}$ encodes the capabilities of the vector field to predict the state at the next time step. This is the MSE of the output of the RK4 scheme and the output of DNN, given as follows:

$$\mathcal{L}_{\text{RK4}} = \frac{1}{\mathcal{N}-1} \sum_{k=1}^{\mathcal{N}-1} \left\| \frac{1}{h_k} (\mathbf{x}(t_{k+1}) - \mathcal{F}_{\text{RK4}}(\Theta(\mathbf{x}(t_k))\Xi, \mathbf{x}(t_k), h_k)) \right\|_2^2. \quad (13)$$

It is worth highlighting that the coefficient matrix Ξ will be updated alongside the weights and biases of the DNN, and the dictionary terms

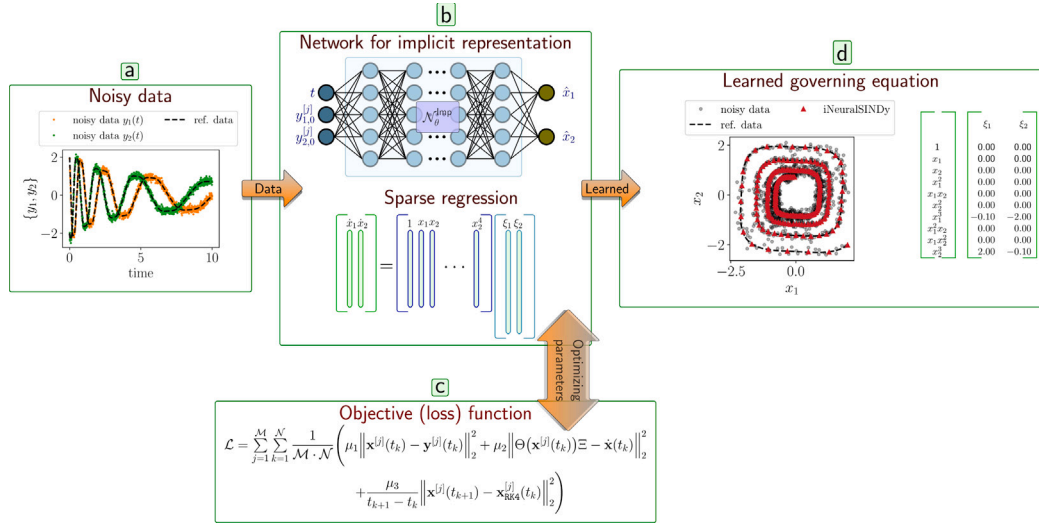


Fig. 1. A schematic diagram of the approach iNeural-SINDy. (a) noisy measurement data, (b) feeding the initial condition $(y_{1,0}, y_{2,0})$ and the time t to the DNN, (c) using the output of the DNN, construct a polynomial dictionary, (d) estimating the parameters of the DNN and sparse vector Ξ by considering a loss function.

Algorithm 2 iNeuralSINDy: SINDy combined with neural network and integral scheme for nonlinear system identification.

Input: Data set $\{y(t_0), y(t_1), \dots, y(t_N)\}$, tol for sequential thresholding, a dictionary containing candidate functions Θ , a neural network G_θ (parameterized by θ), initial iteration (init-iter), maximum iterations max-iter, and parameters $\{\mu_1, \mu_2, \mu_3\}$.

Output: Estimated coefficients Ξ , defining governing equations.

- 1: Initialize the DNN module parameters, and the coefficients Ξ
- 2: $k = 1$
- 3: **while** $k < \text{max-iter}$ **do**
- 4: Feed time t_i as an input to the DNN (G_θ) and predict output $\mathbf{x}$.
- 5: Compute the derivative information $\dot{\mathbf{x}}$ using automatic differentiation.
- 6: Compute the cost function (10).
- 7: Update the parameters of DNN (θ) and the coefficient Ξ using gradient descent.
- 8: **for** $k\%q == 0$ & $k > \text{init-iter}$ **do** ▷ Employing sequential thresholding after q iterations
- 9: $\text{small_inds} = (\text{abs}(\Xi) < \text{tol})$ ▷ identifying small coefficients
- 10: $\Xi(\text{small_inds}) = 0$ ▷ excluding small coefficients
- 11: Update the parameters of DNN (θ) and the coefficient Ξ using gradient descent,
- 12: while ensuring $\Xi(\text{small_inds})$ remains zero.
- 12: $k = k + 1$.

are calculated by (4). Furthermore, after a certain number of epoch training, we employ sequential thresholding on Ξ to remove small coefficients as sketched in Algorithm 1, and update the remaining parameters thereafter. We summarize the procedure in Algorithm 2. Additional steps in Algorithm 2 are as follows. We train our network for initial iterations (denoted by init-iter) without employing sequential thresholding; this helps the DNN to learn the underlying dynamics of the dataset. Afterward, we employ sequential thresholding every q iterations. In the rest of the paper, the proposed methodology is referred to as iNeural-SINDy. We depict a schematic diagram of our proposed approach in Fig. 1.

4. Extension to multi-trajectories data

Thus far, we have presented the discovery of governing equations using a single trajectory time series data set using a single initial condition. However, for complex dynamical processes, a single trajectory is not sufficient to describe underlying dynamics completely. Therefore, it

is necessary to collect data using multiple trajectories; hence, we need to adopt our proposed methodology to account for multiple trajectories.

To achieve this goal, we augment the input time t with an initial condition so that a DNN can capture the nonlinear behavior of the system with respect to different initial conditions. To that end, let us consider $\mathcal{M}$ different trajectories with initial conditions $y_0^{[j]}$, where $j \in \{1, \dots, \mathcal{M}\}$. To reflect the multi-trajectories in our framework, we modify the architecture of the DNN, which now takes t_k and $y_0^{[j]}$ as inputs, and intend to predict $y_k^{[j]}$ —that is, the state at time t_k with respect to the initial condition $y_0^{[j]}$. Then, we also adapt our loss function (10) as follows:

$$\mathcal{L} = \mu_1 \sum_{j=1}^{\mathcal{M}} \mathcal{L}_{\text{MSE}}^{[j]} + \mu_2 \sum_{j=1}^{\mathcal{M}} \mathcal{L}_{\text{deri}}^{[j]} + \mu_3 \sum_{j=1}^{\mathcal{M}} \mathcal{L}_{\text{RK4}}^{[j]}, \quad \mu_1, \mu_2, \mu_3 \in [0, 1], \quad (14)$$

where

$$\mathcal{L}_{\text{MSE}}^{[j]} = \frac{1}{\mathcal{M} \cdot \mathcal{N}} \sum_{j=1}^{\mathcal{M}} \sum_{k=1}^{\mathcal{N}} \left\| y^{[j]}(t_k) - \hat{x}^{[j]}(t_k) \right\|_2^2,$$

$$\mathcal{L}_{\text{deri}}^{[j]} = \frac{1}{\mathcal{M} \cdot \mathcal{N}} \sum_{j=1}^{\mathcal{M}} \sum_{k=1}^{\mathcal{N}} \left\| \Theta(\hat{x}^{[j]}(t_k)) \hat{\Xi} - \dot{\hat{x}}(t_k) \right\|_2^2,$$

$$\mathcal{L}_{\text{RK4}}^{[j]} = \frac{1}{h} \frac{1}{\mathcal{M} \cdot \mathcal{N} - 1} \sum_{j=1}^{\mathcal{M}} \sum_{k=1}^{\mathcal{N}-1} \left\| x^{[j]}(t_{k+1}) - \hat{x}_{\text{RK4}}^{[j]}(t_k) \right\|_2^2 \quad \text{with } h = t_{k+1} - t_k.$$

The ability to handle multiple trajectories in iNeural-SINDy aligns with advancements in promoting group sparsity for discovering governing equations, as highlighted in works like Tod et al. (2021) and Maddu et al. (2021). These studies emphasize leveraging shared structures across datasets through group sparsity to infer consistent dynamics, even when coefficients vary between trajectories. For instance, in Tod et al. (2021) a randomized adaptive group Lasso integrated with neural networks was introduced, enabling robust discovery from noisy, multi-experiment data by enforcing shared dynamics across trajectories. Similarly, in Maddu et al. (2021) it has been demonstrated how group sparsity could encode physical constraints like conservation laws, ensuring robustness and consistency in inferred models under high noise levels.

5. Numerical experiments

In this section, we demonstrate the proposed methodology, the so-called iNeural-SINDy, by means of several numerical examples and present a comparison with existing methodologies. For the comparison, we primarily consider two approaches, namely DeepPyMod (Both

et al., 2021), and RK4-SINDy (Goyal and Benner, 2022). DeePyMoD utilizes only automatic-differential tools to estimate derivative information by constructing an implicit representation of the noisy data, while RK4-SINDy embeds a numerical integration scheme to avoid computation of derivative information. The proposed methodology iNeural-SINDy can be viewed as a combination of DeePyMoD and RK4-SINDy. For the chaotic Lorenz example, we also present a comparison with Weak-SINDy (Messinger and Bortz, 2021).

To quantify the performance of the considered methodologies, we define the following coefficient error measure for each state variable x_i :

$$\mathcal{E}(x_i) = \left\| \Xi_{x_i}^{\text{truth}} - \Xi_{x_i}^{\text{est}} \right\|_1, \quad (15)$$

where $\Xi_{x_i}^{\text{truth}}$ and $\Xi_{x_i}^{\text{est}}$ are, respectively, the true and estimated coefficients, corresponding to the state variable x_i , and $\|\cdot\|_1$ denotes the l_1 -norm. A motivation to quantify each state variable separately is that their dynamics can be of different scales; thus, their coefficients might also be of different order. Therefore, to better understand the quality of the discovered models, we analyze them separately. Furthermore, to observe the performance of the methodologies under noisy data, which is often the case in real-world scenarios, we artificially generate noisy data by corrupting the clean data. For this, we use a white Gaussian noise $\mathcal{N}(\mu, \sigma^2)$ with a zero mean $\mu = 0$ and variance σ^2 , where σ denotes the standard deviation. The noise level in the data is controlled by σ , i.e., larger σ implies more noise present in the data. Additionally, since iNeural-SINDy and DeePyMoD both involve neural networks, we also compare their performance sensitivity in two scenarios as follows:

- **Scene_A:** In the first scenario, we consider having a single initial condition and a fixed number of neurons in the hidden layers but vary the amount of training data and noise levels.
- **Scene_B:** In the second one, we consider having a single initial condition and a fixed number of training data but vary the number of neurons in the hidden layers and noise levels.

In addition, in the following, we further clarify common implementation and reproducibility details that are considered for all the examples.

Data generation. We have generated the data synthetically by using `solve_ivp` function from `scipy.integrate` package to solve a given set of differential equations and produce the data set. When an identification approach terminates based on the considered methodologies (e.g., iNeural-SINDy, DeePyMoD, RK4-SINDy, or Weak-SINDy), we multiply the dictionary Θ by estimated coefficient matrix Ξ^{est} to obtain the discovered governing equations. We then make use of the `solve_ivp` function from `scipy.integrate` to obtain time-evolution dynamics.

Moreover, we perform a data-processing step before feeding to a neural network by mapping the minimum and maximum values to -1 and 1 , respectively. The hyper-parameters μ in (14) are set to $\mu_1 = 1$, $\mu_2 = 0.1$ and $\mu_3 = 0.1$ for iNeural-SINDy. Note that we can drive RK4-SINDy and DeePyMoD approaches by setting $\mu_3 = 0$ and $\mu_2 = 0$, respectively, in (14).

Architecture. We use multi-layer networks with periodic activation functions, namely, SIREN (Sitzmann et al., 2020), to learn an implicit representation based measurement data. We choose SIREN activation because its infinite differentiability facilitates accurate derivative-based computations (unlike ReLU's piecewise linear structure), and previous studies such as Goyal and Benner (2022) and Both et al. (2021) have shown its superiority compared to the ReLU activation function. The numbers of hidden layers and neurons will be discussed for each example separately. However, we typically opt three or four hidden layers to strike a balance between expressive power and computational cost. These sine-based activations help capture complex, wave-like

solution structures more effectively than standard ReLU or tanh in certain scenarios as demonstrated in Sitzmann et al. (2020).

Hardware. For training neural networks and parameter estimations for discovering governing equations, we have used NVIDIA[®] RTX A4000 GPU with 16 GB RAM, and for CPU computations (e.g., for generating data), we have used a 12th Gen Intel[®] Core™-12600K processor with 32 GB RAM.

Training set-up. We use the Adam optimizer (Kingma and Ba, 2014) to update the coefficient matrix Ξ that is trained alongside the DNN parameters. In other words, we perform end-to-end training where both the DNN parameters (weights and biases) and the SINDy coefficients are optimized via an Adam optimizer. As mentioned earlier, the iNeuralSINDy loss function is composed of a weighted combination of three terms mismatch between the neural network outputs and the (noisy) measured data, derivative alignment (SINDy), and one-step-ahead integral consistency (RK4-SINDy)-jointly enforcing denoised trajectory learning and sparse system identification. The threshold value (`tol`), learning rate of the optimizer, maximum iterations (`max-iter`), initial iterations (`init-iter`), the iteration q for employing sequential thresholding for Algorithm 2 will be mentioned for each example separately. However, we note that after each thresholding step in Algorithm 2, we reset the learning rate 5×10^{-6} for DNN parameters and 1×10^{-2} for the coefficient matrix Ξ^{est} except for the Lorenz example, which is explicitly mentioned in the Lorenz example.

We primarily selected the learning rates and threshold values based on empirical observations and prior experience with similar dynamical systems. In a more rigorous study, one would typically employ systematic hyperparameter optimization techniques (e.g., grid search (Bergstra and Bengio, 2012), Bayesian optimization (Feurer and Hutter, 2019), or cross-validation (Arlot and Celisse, 2010)) to fine-tune these parameters. Moreover, it is worth noting that the optimal hyperparameters can depend significantly on the chosen optimizer (e.g., Adam vs. stochastic gradient descent), further motivating a methodological tuning approach in practice.

We follow the same sparsity-enforcing procedure introduced in the original SINDy framework, which uses sequential thresholding rather than an explicit ℓ_1 -penalty term during optimization. Specifically, the coefficients are pruned by iteratively setting small values to zero, as described in Brunton et al. (2016a). This approach ensures a computationally efficient implementation of sparsity without directly incorporating an ℓ_1 -penalty into the gradient descent process. Furthermore, we employed this strategy to maintain consistency with the methodologies used in other comparable algorithms, such as DeePyMoD (Both et al., 2021) and RK4-SINDy (Goyal and Benner, 2022), enabling a fair comparison of results.

Noise level setup. We choose noise levels (ranging from low to relatively high percentages of the signal amplitude) that are representative of mild through severe real-world measurement corruption, thereby allowing us to demonstrate the robustness and effectiveness of our method across a realistic spectrum of noisy conditions compared to RK4-SINDy and DeePyMoD. For each example we specify the noise level separately.

Case studies. We select a diverse array of example oscillators to evaluate the iNeuralSINDy approach, each chosen for their distinct characteristics and relevance to different areas of dynamical systems analysis. The two-dimensional Damped and Cubic Oscillators serve as simple, low-dimensional systems with known solutions, ideal for visualizing trajectories and verifying the accuracy of learned coefficients (Strogatz, 2018). The FitzHugh-Nagumo System, a standard model in computational neuroscience, represents nonlinear and oscillatory dynamics typical of excitable media like neuron activity, and is often used to test system identification methods in biological settings (Izhikevich, 2007). Lastly, the Lorenz System, a prototype

of chaotic behavior, tests the method's ability to capture and predict dynamics that are highly sensitive to initial conditions, making it a benchmark in studies of chaos within data-driven discovery frameworks (Sprott and Sprott, 2003).

5.1. Two-dimensional damped oscillators

In our first example, we consider the discovery of a two-dimensional linear oscillatory damped system using data. The dynamics of the oscillator can be given by

$$\begin{aligned}\dot{x}_1(t) &= -0.1x_1(t) + 2.0x_2(t), \\ \dot{x}_2(t) &= -2.0x_1(t) - 0.1x_2(t).\end{aligned}\quad (16)$$

Simulation setup. To generate the training data set, we consider three initial conditions in the range $[-2, 2]$ for x_1 and x_2 , and for each initial condition, we take 400 equidistant points in the time interval $t \in [0, 10]$. Our DNN architecture has three hidden layers, each having 32 neurons. We set the number of epochs $\text{max-iter} = 15,000$ and threshold value $\text{tol} = 0.05$. The initial iteration init-iter is set to 5,000 with the learning rate of 10^{-4} for the DNN parameters and 10^{-3} for the coefficient matrix Ξ^{est} , and after $q = 2,000$ iterations, we employ the sequential thresholding. Moreover, we construct a dictionary containing polynomials of degree up to two.

Results. Fig. 2 demonstrates the performance of different algorithms in the presence of noise. We consider additive white Gaussian noise with different standard variances $0 \leq \sigma \leq 0.65$. As demonstrated in Fig. 2, as we increase the noise level, the RK4-SINDy fails to estimate the coefficients. For $\sigma \leq 0.45$, iNeural-SINDy and DeePyMoD are robust in discovering the underlying equations accurately, even for high noise levels, and both exhibit similar performance. However, for $\sigma \geq 0.5$ DeePyMoD loses its performance compared with iNeural-SINDy. In Table A.1 (in the appendix), we also report learned governing equations from data with various noise levels, which again illustrate that both iNeural-SINDy and DeePyMoD have similar performance, and RK4-SINDy fails to recover governing equations from highly noisy data. Furthermore, in Fig. 3, the error in the convergence of the non-zero coefficients for the different methods is shown as the training progresses. It can be seen from Fig. 3 that iNeural-SINDy has a faster convergence rate compared to DeePyMoD and RK4-SINDy. Next, we discuss the performance of iNeural-SINDy and DeePyMoD for Scene_A and Scene_B.

- **Scene_A:** We consider a DNN architecture with three hidden layers, each having 32 neurons. For comparison, we consider noise levels with standard variance $\sigma = \{0, 0.02, 0.04, 0.06\}$, and take the number of samples $\{30, 40, 50, 100, 200, 300, 400\}$ in the time interval $[0, 10]$ for a single initial condition $(x_1(0), x_2(0)) = (5, 2)$. The rest of the settings are the same as mentioned earlier in the simulation setup. By varying the noise levels and the number of samples, we report the quality of the learned governing equations in Fig. 4. Note that the error criterion defined in (15) is used. Each cell shows the error corresponding to sample sizes and noise levels. By comparing the simulation results, we notice that DeePyMoD performs better for low data regime, but as the number of data is increased, both iNeural-SINDy and DeePyMoD perform similarly.
- **Scene_B:** In this case, we consider a DNN architecture with three hidden layers but vary the number of neurons at each layer from 2 to 64. Again, we consider various noise levels. We take 400 samples in the time interval $[0, 10]$ for a single arbitrary initial condition $(x_1(0), x_2(0)) = (5, 2)$. The rest of the settings are the same as mentioned earlier in the simulation setup. By varying the noise levels and number of neurons, we report a comparison between iNeural-SINDy and DeePyMoD in Fig. 5, where each cell shows the error, corresponding to a specific number of

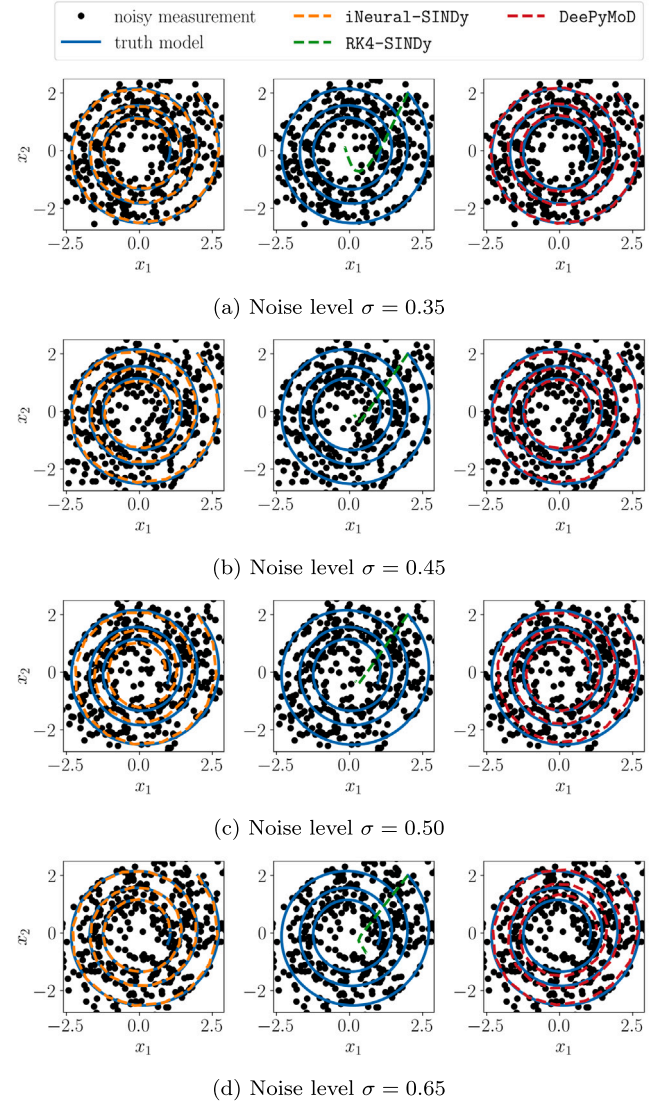


Fig. 2. Linear oscillator: A comparison of the learned equations using different methods under various noise levels present in measurements, compared with the ground truth. As the noise level increases, RK4-SINDy loses its performance; however, iNeural-SINDy and DeePyMoD maintain competitive performance.

neurons and noise level. These comparisons again show that both methodologies perform comparably and learn correct coefficients with similar performance for a large number of neurons as the DNN has more capacities to capture the dynamics present in the data. More interesting, we would like to highlight that both methods do not over-fit as the capacity of the DNN is increased.

5.2. Cubic damped oscillator

The cubic oscillatory system is given by the following equation:

$$\begin{aligned}\dot{x}_1(t) &= -0.1x_1^3(t) + 2.0x_2^3(t), \\ \dot{x}_2(t) &= -2.0x_1^3(t) - 0.1x_2^3(t).\end{aligned}\quad (17)$$

The system consists of two coupled, non-linear differential equations describing the time evolution of two variables, x_1 and x_2 . Given noisy data, we aim to recover the governing equations and perform a similar analysis as done for the previous example.

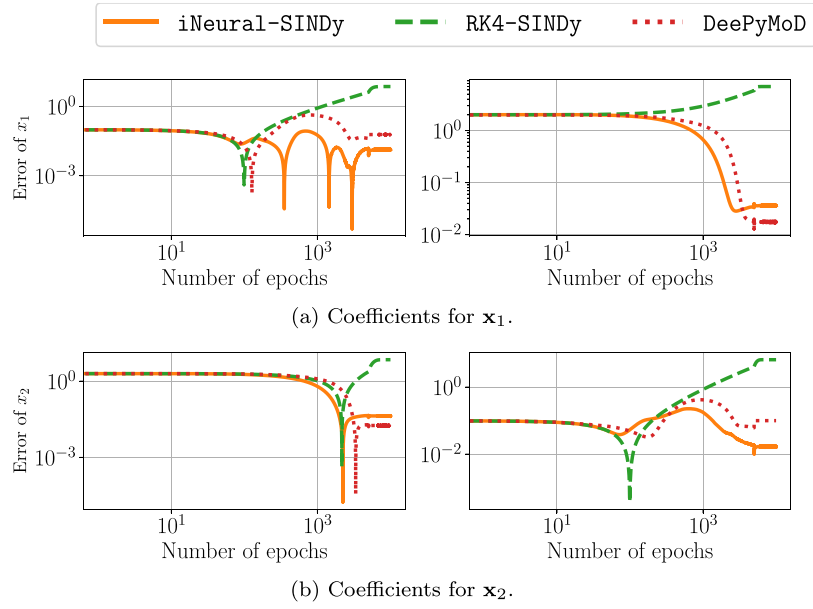


Fig. 3. Linear oscillator: Error of estimated coefficients (see (15)) during the training loop for iNeural-SINDy, DeePyMoD, and RK4-SINDy when noise level is $\sigma = 0.65$. The behavior of states x_1 and x_2 are shown for the coefficients corresponding to the terms $\{x_1, x_2\}$ in the dictionary Θ as training progress.

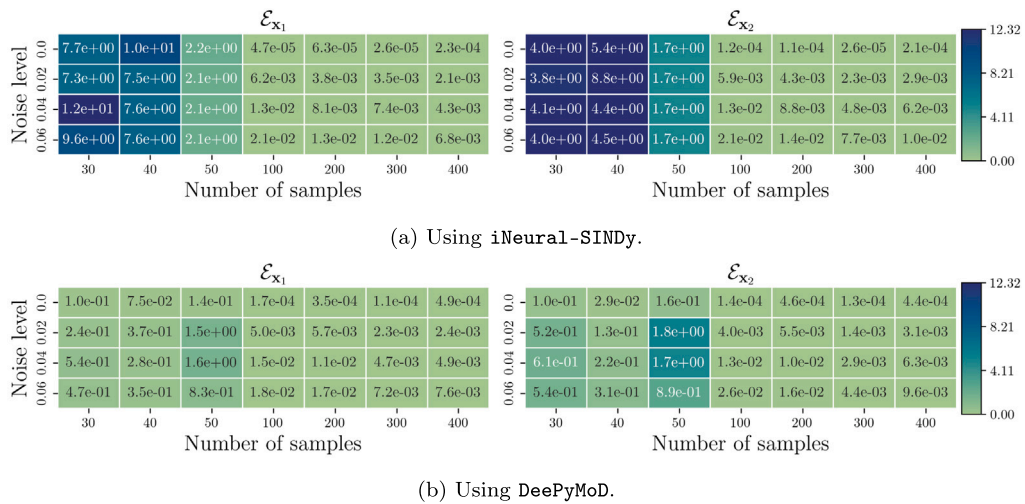


Fig. 4. Linear oscillator: A comparison of iNeural-SINDy and DeePyMoD under Scene_A.

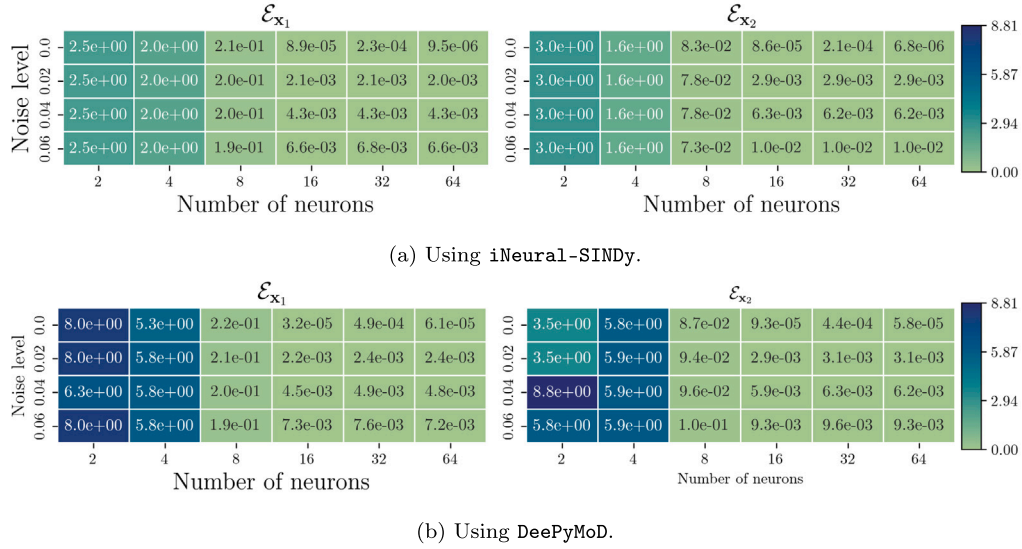
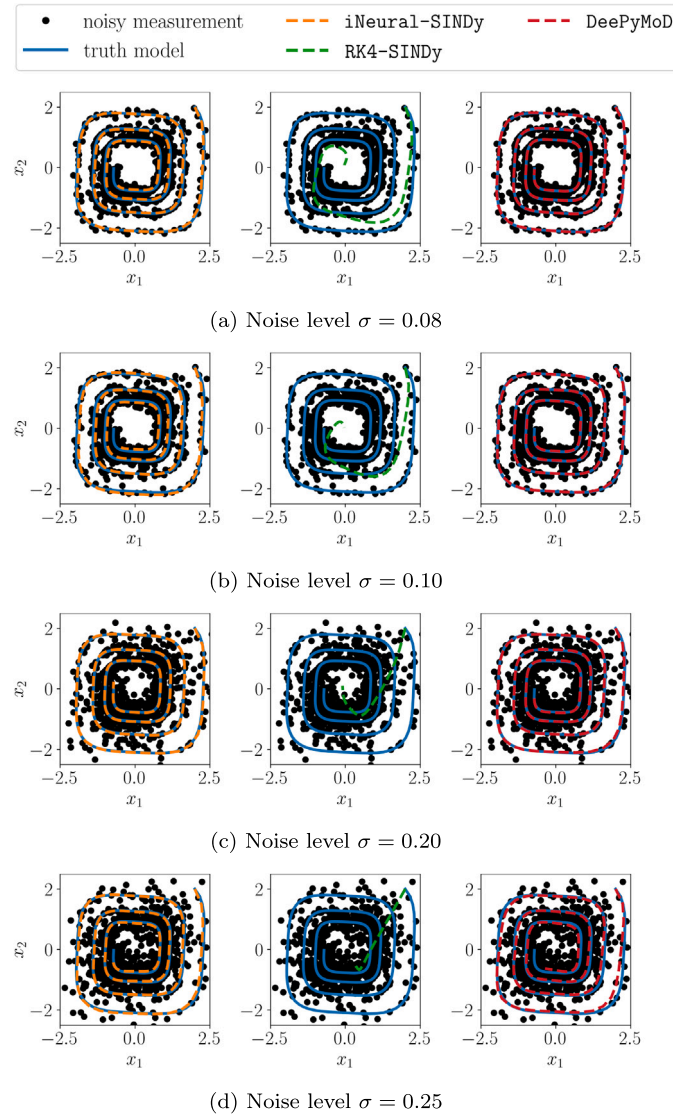


Fig. 5. Linear oscillator: A comparison of iNeural-SINDy and DeePyMoD under Scene_B.

Fig. 6. Cubic damped oscillator: A comparison of the estimation with different techniques and noise level. As noise level increases RK4-SINDy loses its performance, however iNeural-SINDy and DeePyMoD have competitive performance. However as reported in Table A.3 for higher value of noise ($0.08 \leq \sigma \leq 0.25$) DeePyMoD loses its performance.

simulation setup. To generate the training data set, we consider two initial conditions $(\mathbf{x}_1(0), \mathbf{x}_2(0)) = \{(2, 2), (-2, -2)\}$ and collect 800 points in the time interval $t \in [0, 10]$. Our DNN architecture has three hidden layers, each having 32 neurons. We set the number of epoch `max-iter` = 30,000, threshold value `tol` = 0.05. The initial training iteration (`init-iter`) is set to 15,000 with the learning rate 10^{-4} for the DNN parameters and 10^{-3} for the coefficient matrix Ξ^{est} . After the initial training, for every $q = 5,000$ iterations afterward, we employ the sequential thresholding and update the DNN parameters and Ξ^{est} . The dynamical system is estimated in the space of polynomials up to order three.

Results. To see the performance of these different methodologies under the presence of noise, we consider Gaussian noise with the standard deviation $0 \leq \sigma \leq 0.35$. We report the obtained results in Fig. 6, Fig. 7 and in Table A.2 (see Appendix), and we notice that RK4-SINDy performs poorly for high noise levels, but iNeural-SINDy and DeePyMoD have competitive performance up to the noise level $\sigma \leq 0.08$. However, as noise level increases to higher values $\sigma \geq 0.08$ iNeural-SINDy outperforms DeePyMoD. This difference is more evident in the number of non-zero elements in the coefficients corresponding to noise level $0.2 \leq \sigma \leq 0.35$ for DeePyMoD in Table A.2. Further, in Fig. 8, we plot the error in convergence of the non-zero coefficients as the training progresses for the noise-free case. Here, we again observe a faster convergence for iNeural-SINDy as compared to the other two approaches. Next, we investigate performances of iNeural-SINDy and DeePyMoD for Scene_A and Scene_B, which are discussed in the following.

- **Scene_A:** We fix a DNN architecture with three hidden layers, each having 32 neurons. We consider a set of noise level with $\sigma = \{0, 0.02, 0.04, 0.06\}$ and a set of sample size $\{30, 40, 50, 100, 200, 300, 400\}$. The data are collected using a random initial condition in the interval $[1, 4]$ for $\{\mathbf{x}_1, \mathbf{x}_2\}$. The rest of the settings are the same as mentioned earlier in the simulation setup. The results are shown in Fig. 9, where we notice that for a smaller data set, iNeural-SINDy performs slightly better as compared to DeePyMoD, whereas for larger data set, it is otherwise.
- **Scene_B:** For this case, we fix the sample size to 400 but consider a DNN architecture with three hidden layers with the number of neurons ranging from 2 to 64. Furthermore, we consider a set of noise levels with $\sigma = \{0, 0.02, 0.04, 0.06\}$. The data are generated as in Scene_A and the training setting is also as above. The results are depicted in Fig. 10, where we observe that iNeural-SINDy performs better as compared to DeePyMoD for fewer neurons, and as we increase the number of neurons, both methods perform similarly.

5.3. Fitz-Hugh Nagumo system

The Fitz-Hugh Nagumo (FHN) model is a non-linear system of ordinary differential equations that is used to describe the behavior of a biological neuron, see, e.g., Schaeffer and McCalla (2017). The FHN model is commonly used to study the behavior of biological neurons under different conditions, such as the changes in the external stimulus or variations in the intrinsic properties of the neuron. The set of differential equations that describe the underlying dynamics are as follows:

$$\begin{aligned}\dot{\mathbf{x}}_1(t) &= 1.0\mathbf{x}_1(t) - 1.0\mathbf{x}_2(t) - \frac{1}{3}\mathbf{x}_1^3(t) + 0.1, \\ \dot{\mathbf{x}}_2(t) &= 0.1\mathbf{x}_1(t) - 0.1\mathbf{x}_2(t).\end{aligned}\quad (18)$$

Simulation setup. For this simulation example, we consider two initial conditions $(\mathbf{x}_1(0), \mathbf{x}_2(0)) = \{(2, 1.5), (1.5, 2)\}$ and take 400 data points in the time interval $t \in [0, 200]$. The DNN architecture has three hidden layers with 32 neurons. We set the number of epoch `max-iter` =

50,000 and threshold value `tol` = 0.05. The number of iterations for the initial training is set to 15,000 with the learning rate 10^{-4} for the DNN parameters and 10^{-3} for the coefficient matrix Ξ^{est} . After the initial training, we employ the sequential thresholding after each $q = 5,000$ iterations. We aim to learn the underlying governing equations in the space of polynomials with degree up to order three.

Results. Converse to the results in the previous two examples, for the FHN, iNeural-SINDy has a slower convergence rate compared to DeePyMoD and RK4-SINDy, see Fig. 11 for tracking the evolution of different coefficients through the training iterations.

For this example, we again make a similar observation (see Fig. 12, and Table A.3 in Appendix), where we notice that iNeural-SINDy and DeePyMoD exhibit similar performances for lower noise levels, but for the higher noise values, (see the results for $\sigma \geq 0.08$ Table A.3, iNeural-SINDy tends to outperform DeePyMoD. It is worth to highlight that for noise level $\sigma \geq 0.15$ DeePyMoD yields unstable system (see Fig. 12(e)). Moreover, RK4-SINDy clearly fails for high noise levels. However, converse to the results reported in the previous two examples, for this example, we notice that it is hard to judge about the convergence rate of iNeural-SINDy compared to DeePyMoD and RK4-SINDy; see, Fig. 11. But we highlight that iNeural-SINDy can identify governing equations for highly noisy data, as stated earlier. Next, we compare the performances of iNeural-SINDy and DeePyMoD under Scene_A and Scene_B.

- **Scene_A:** In this case, we consider a fixed DNN architecture with three hidden layers, each consisting of 32 neurons. The different noise levels $\{0.0, 0.02, 0.04, 0.06\}$ are considered, while the sample size is considered in the range from 150 to 450 with an increment of 50. Here, a single initial condition is used for data collection; that is, $(\mathbf{x}_1(0), \mathbf{x}_2(0)) = (3, 2)$. The rest of the settings are the same as mentioned earlier for this example. The results are shown in Fig. 13, where we notice that DeePyMoD outperforms iNeural-SINDy and has a better performance.
- **Scene_B:** Here, we conduct a study where we keep the number of samples fixed at 400, obtained using the initial condition $(\mathbf{x}_1(0), \mathbf{x}_2(0)) = (3, 2)$. The DNN architecture is designed to have three hidden layers. We aim to explore how iNeural-SINDy and DeePyMoD perform under different combinations of neurons for each layer and noise level. The training settings for each case remain the same, as mentioned earlier. The outcomes are presented in the heat-map depicted in Fig. 14, where we notice that both DeePyMoD and iNeural-SINDy almost have the same performance in all the settings.

5.4. Chaotic Lorenz system

The chaotic Lorenz system is a set of three differential equations as follows (Kuznetsov and Reitmann, 2020):

$$\dot{\mathbf{x}}_1(t) = \gamma(\mathbf{x}_2(t) - \mathbf{x}_1(t)), \quad (19a)$$

$$\dot{\mathbf{x}}_2(t) = \mathbf{x}_1(t)(\rho - \mathbf{x}_3(t)) - \mathbf{x}_2(t), \quad (19b)$$

$$\dot{\mathbf{x}}_3(t) = \mathbf{x}_1(t)\mathbf{x}_2(t) - \beta\mathbf{x}_3(t), \quad (19c)$$

where the parameters γ , ρ , and β are positive constants with associated standard values $\gamma = 10$, $\rho = 28$, $\beta = \frac{8}{3}$. The Lorenz system is a classic example of a chaotic system, which means that small differences in the initial conditions can lead to vastly different outcomes over time. It is a widely used benchmark example for discovering governing equations (Brunton et al., 2016a).

Simulation setup. We collect our data in the time interval $t \in [0, 10]$ with a sample size of 200 for three different initial conditions $(\mathbf{x}_1(0), \mathbf{x}_2(0), \mathbf{x}_3(0)) = \{(-8, 7, 27), (-6, 6, 25), (-9, 8, 22)\}$. The DNN architecture has three hidden layers, each having 64 neurons. We set the

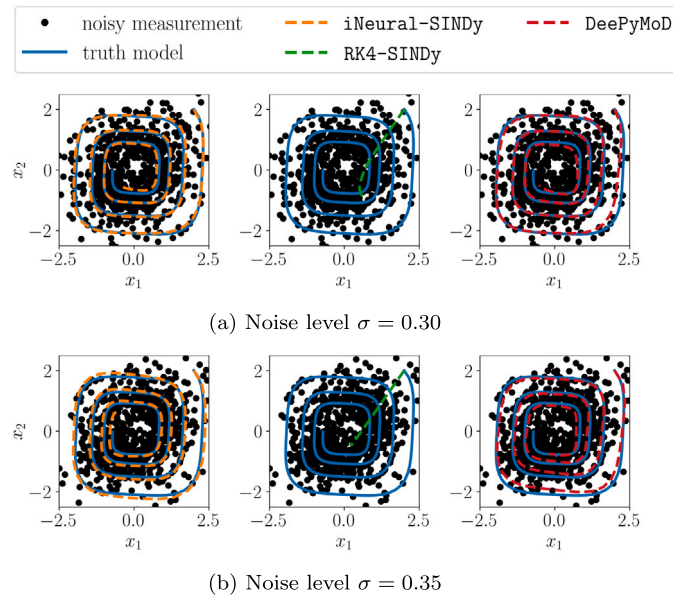


Fig. 7. Cubic damped oscillator: A comparison of the estimation with different techniques and noise levels. As the noise level increases, RK4-SINDy loses its performance; however, iNeural-SINDy and DeePyMoD maintain competitive performance. As reported in Table A.3, for higher noise levels ($0.3 \leq \sigma \leq 0.35$), DeePyMoD loses its performance.

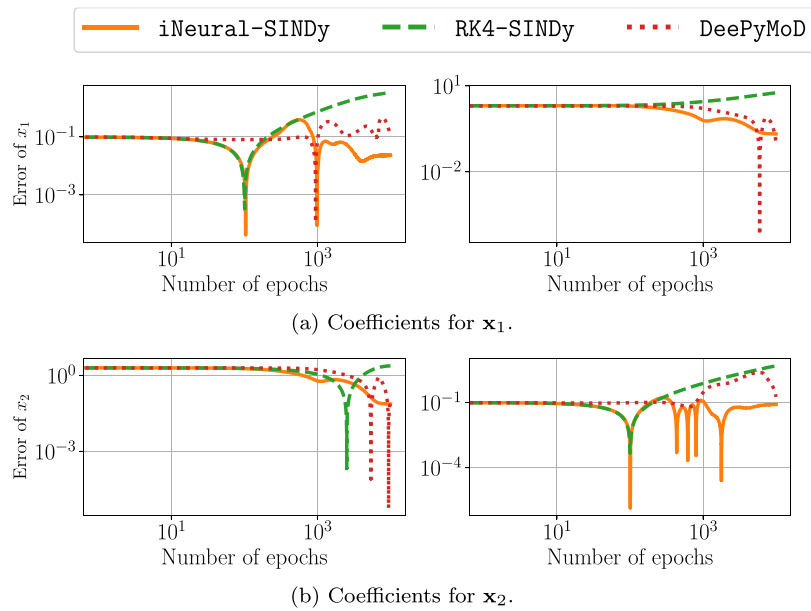
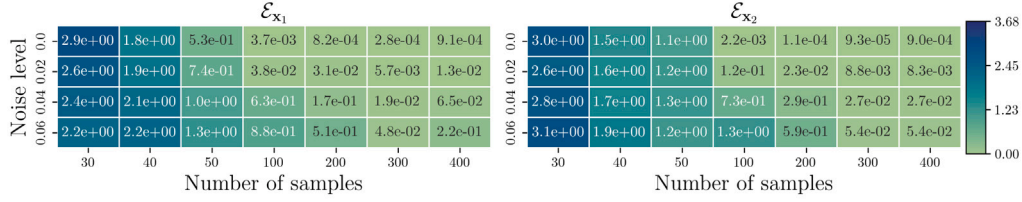
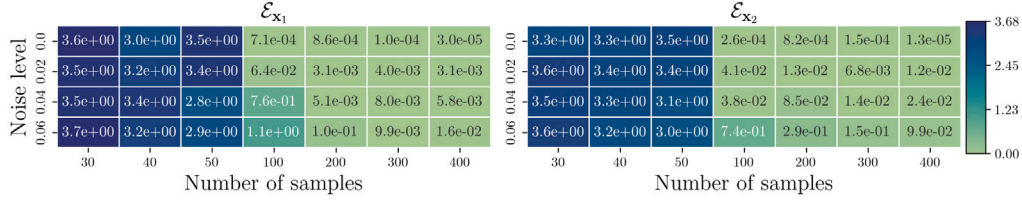


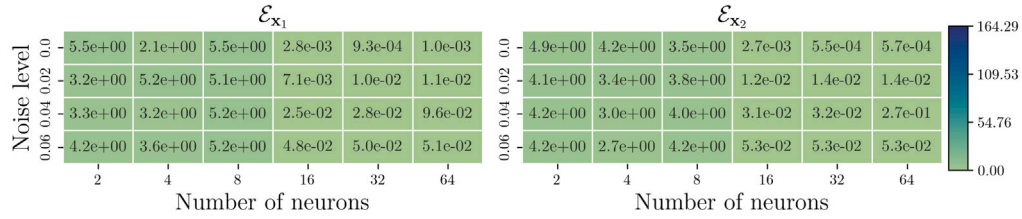
Fig. 8. Cubic oscillator: Error of estimated coefficients (see Eq. (15)) during the training loop for iNeural-SINDy, DeePyMoD and RK4-SINDy when noise level is $\sigma = 0.35$. The behavior of states $\mathbf{x}_1$ and $\mathbf{x}_2$ are shown for the coefficients corresponding to the terms $\{\mathbf{x}_1^3, \mathbf{x}_2^3\}$ in the dictionary Θ as training progress.



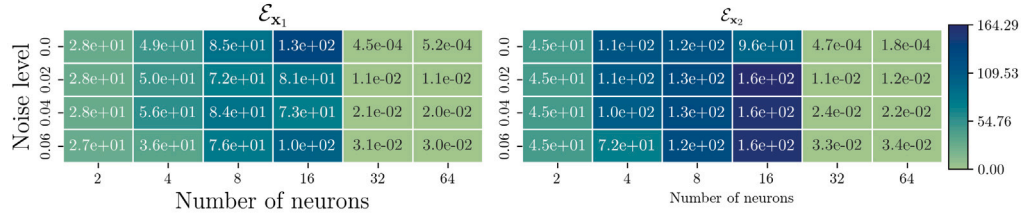
(a) Using iNeural-SINDy.



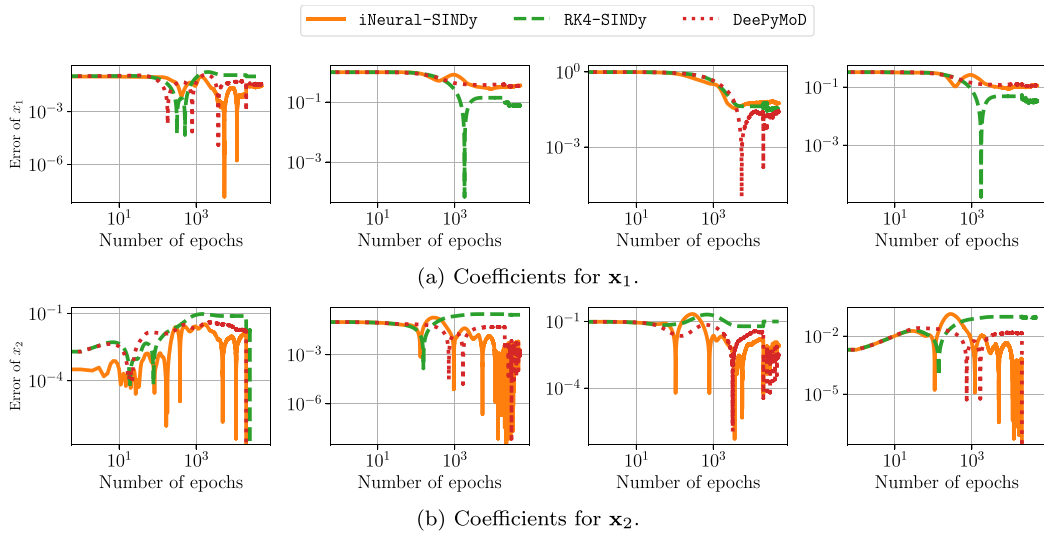
(b) Using DeePyMoD.

Fig. 9. Cubic oscillator: A comparison of iNeural-SINDy and DeePyMoD under Scene_A.

(a) Using iNeural-SINDy.



(b) Using DeePyMoD.

Fig. 10. Cubic oscillator: A comparison of iNeural-SINDy and DeePyMoD under Scene_B.(b) Coefficients for x_2 .**Fig. 11.** Fitz-Hugh Nagumo: Error of estimated coefficients (see Eq. (15)) during the training loop for iNeural-SINDy, DeePyMoD and RK4-SINDy when noise level is $\sigma = 0.12$. The behavior of states x_1 and x_2 are shown for the coefficients corresponding to the terms $\{1, x_1, x_2, x_1^3\}$ in the dictionary Θ as training progress.

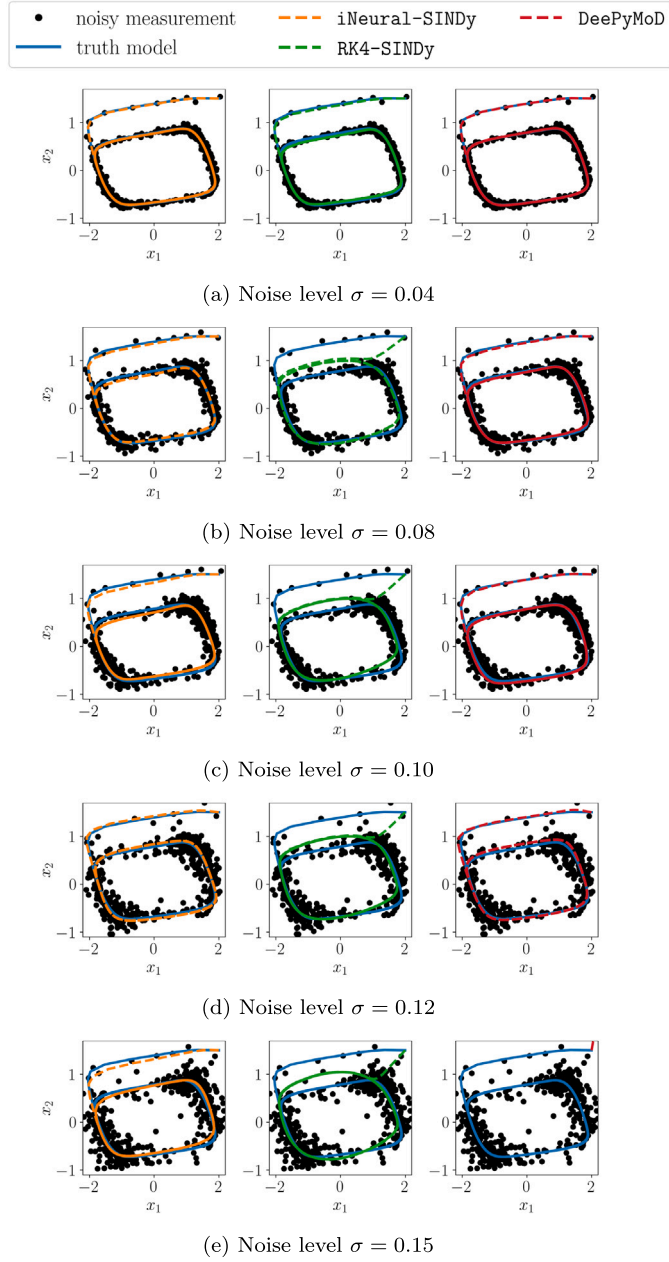


Fig. 12. Fitz-Hugh Nagumo: Comparison of the estimation with different techniques and noise levels. As the noise level increases, RK4-SINDy loses its performance; however, iNeural-SINDy and DeePyMoD maintain competitive performance.

number of iterations $\text{max-iter} = 35,000$, and threshold value $\text{tol} = 0.2$. We set the number of iterations for the initial training to $\text{init-iter} = 10,000$, and the learning rate $7 \cdot 10^{-4}$ for the DNN parameters and 10^{-2} for the coefficient matrix Ξ^{est} . After finishing the initial iterations, we employ sequential thresholding after every $q = 3,000$ iterations. Moreover, after each sequential thresholding, we reset the learning rate for DNN parameters to $5 \cdot 10^{-6}$ and for the coefficient matrix Ξ^{est} to 10^{-2} . The governing equations are estimated by constructing a dictionary with polynomials up to degree two.

Since the magnitude of $\{x_1, x_2, x_3\}$ for the Lorenz example can be large, we consider scaling the x 's using a scaling factor α . Note that

such scaling does not affect the interaction between different x 's; thus, the sparsity pattern remains the same as well. However, it is observed that improving the condition number of the dictionary matrix enhances the estimate of the coefficients and helps us to determine the right governing equations.

Results. We conduct experiments using a scaling factor $\alpha = 0.1$. Further, we aim to learn governing equations from the noisy data with noise levels of $\sigma = \{0, 0.04, 0.1, 0.2, 0.4\}$. We report the obtained results in Table A.4, where we notice that iNeural-SINDy and DeePyMoD yield similar performance except for the case of higher noise level (e.g., see the results for $\sigma = 0.4$), where iNeural-SINDy recovered the equations better. However, RK4-SINDy performs poorly for the higher noise levels. Next, we conduct a performance analysis of iNeural-SINDy and DeePyMoD for Scene_A and Scene_B. We note that in both scenarios, the training data are generated using a single initial condition $(x_1(0), x_2(0), x_3(0)) = (-8, 7, 27)$.

- **Scene_A:** We compare iNeural-SINDy and DeePyMoD under Scene_A. We also investigate the effect of the scaling factor α and consider two values of it, i.e., $\alpha = \{0.1, 1\}$. We fix the DNN architecture to have three hidden layers, each having 64 neurons. We consider different sample sizes and noise levels to compare the performance of iNeural-SINDy and DeePyMoD. For $\alpha = 0.1$, we show the results in Fig. 15, where we notice that iNeural-SINDy outperforms DeePyMoD in most cases. A similar observation is made for $\alpha = 1$, which is reported in Fig. 16. For these experiments, it is hard to conclude the effect of the scaling factors, as we notice that in some cases, the scaling improves the performance, and in some cases, it is not the case.
- **Scene_B:** In this case, we fix the sample size to 400. We also fix the number of hidden layers for the DNN architecture to three but vary the number of neurons in each layer. We also conduct experiments to see the effect of the scaling factor in this case as well. The results for $\alpha = 1$ and $\alpha = 0.1$ are shown in Fig. 17 and Fig. 18, respectively. These heat maps indicate the outperformance of iNeural-SINDy in most cases. We also observe that for a larger number of neurons, the scaling factor $\alpha = 0.1$ slightly performs better compared to scaling factor $\alpha = 1$ in both iNeural-SINDy as well as DeePyMoD.

A comparison of iNeural-SINDy with WEAK-SINDy. Beside our previous comprehensive study, we next compare iNeural-SINDy with Weak-SINDy, which also does not require any estimate of derivatives using noisy data; for more details on Weak-SINDy, we refer to Messenger and Bortz (2021).

For this study, we again consider the same initial condition as used for Scene_A and Scene_B. We take 2000 data points in the time interval $[0, 10]$, which are corrupted using different noise levels $\sigma = \{0, 0.02, 0.08, 0.1\}$. To discover the governing equations, we consider a dictionary of polynomials up to degree two. For training iNeural-SINDy we use the same setting as discussed in Section 5.4. For Weak-SINDy, we consider the code provided by the authors.¹ We report the results in Table A.5, which indicates that iNeural-SINDy outperforms Weak-SINDy in the presence of high noise.

Non-smooth behavior in the coefficient-error plots. The observed non-smooth behavior in the coefficient-error plots (e.g., Fig. 3, Fig. 8, and Fig. 11) is primarily a consequence of the adaptive learning rate used by the Adam optimizer, which internally adjusts step sizes based on momentum and recent gradient history. Consequently, individual updates can vary in magnitude, causing the error curves to fluctuate instead of decreasing smoothly. This effect may be further amplified

¹ https://github.com/MathBioCU/WSINDy_ODE/tree/master

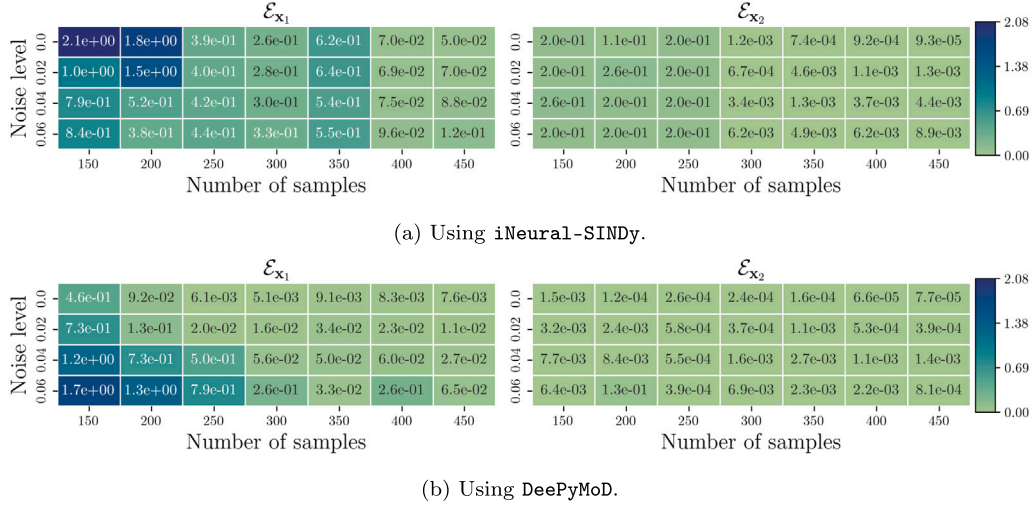


Fig. 13. Fitz-Hugh Nagumo: A comparison of iNeural-SINDy and DeePyMoD under Scene_A.

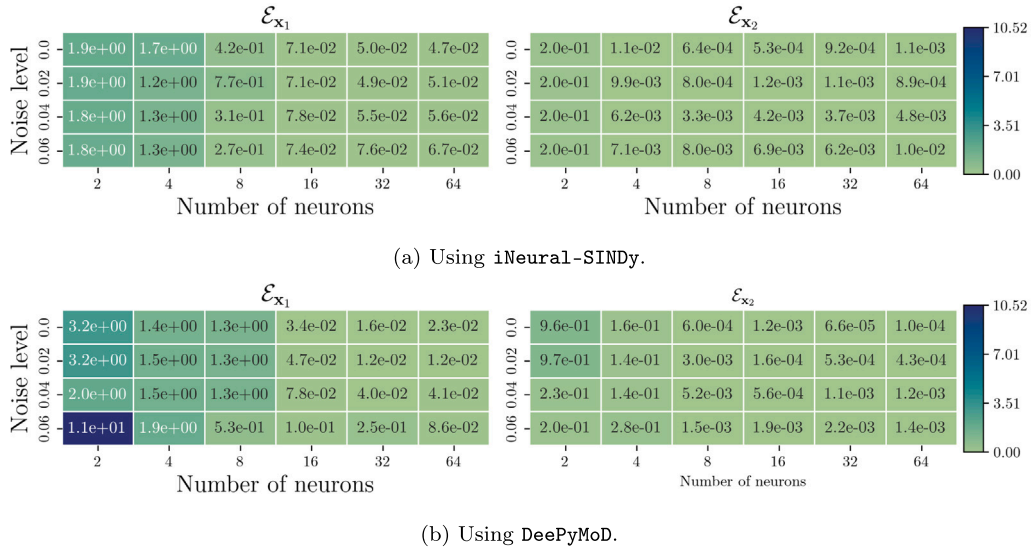
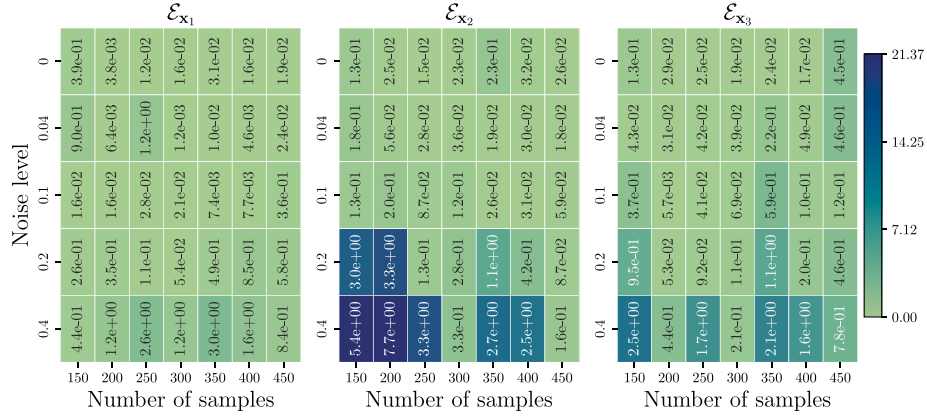
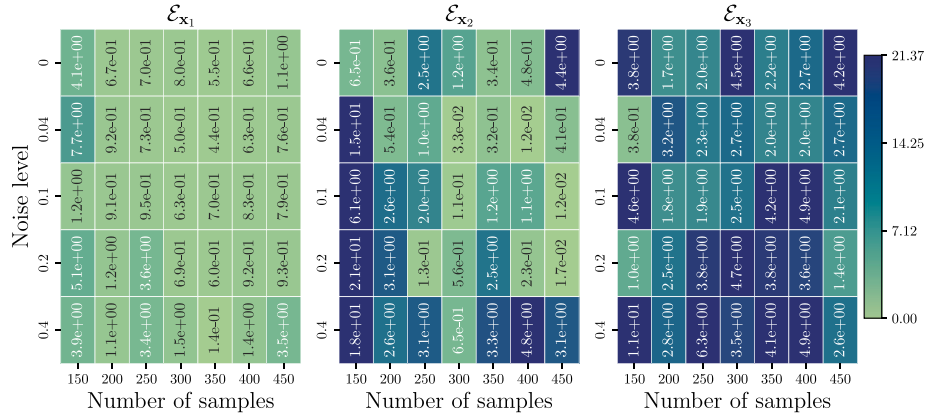


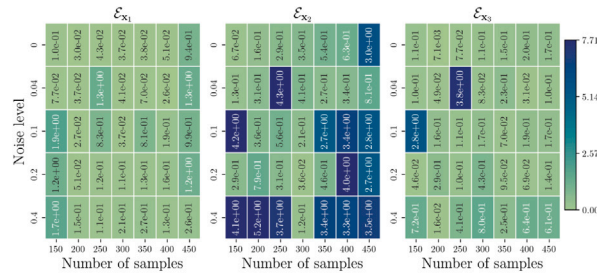
Fig. 14. Fitz-Hugh Nagumo: A comparison of iNeural-SINDy and DeePyMoD under Scene_B.



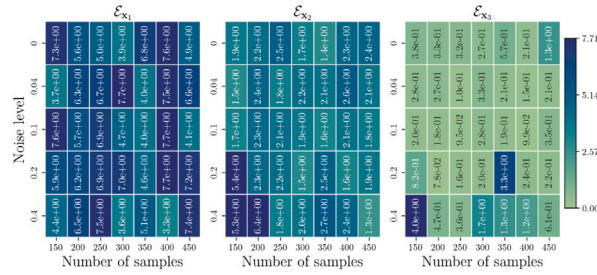
(a) Using iNeural-SINDy.



(b) Using DeePyMoD.

Fig. 15. Lorenz example: A comparison of iNeural-SINDy and DeePyMoD under Scene_A with the scaling factor $\alpha = 0.1$.

(a) Using iNeural-SINDy.



(b) Using DeePyMoD.

Fig. 16. Lorenz example: A comparison of iNeural-SINDy and DeePyMoD under Scene_A with the scaling factor $\alpha = 1$.

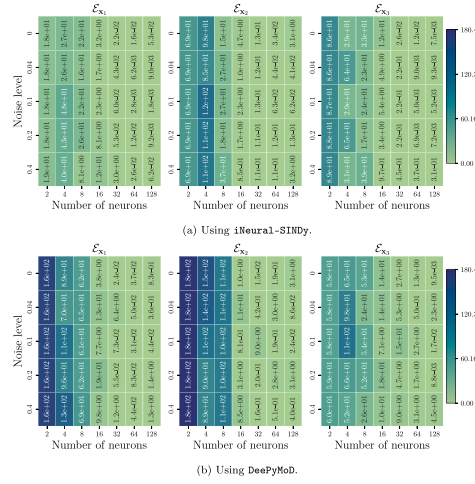


Fig. 17. Lorenz example: A comparison of iNeural-SINDy and DeePyMoD under Scene_B with the scaling factor $\alpha = 0.1$.

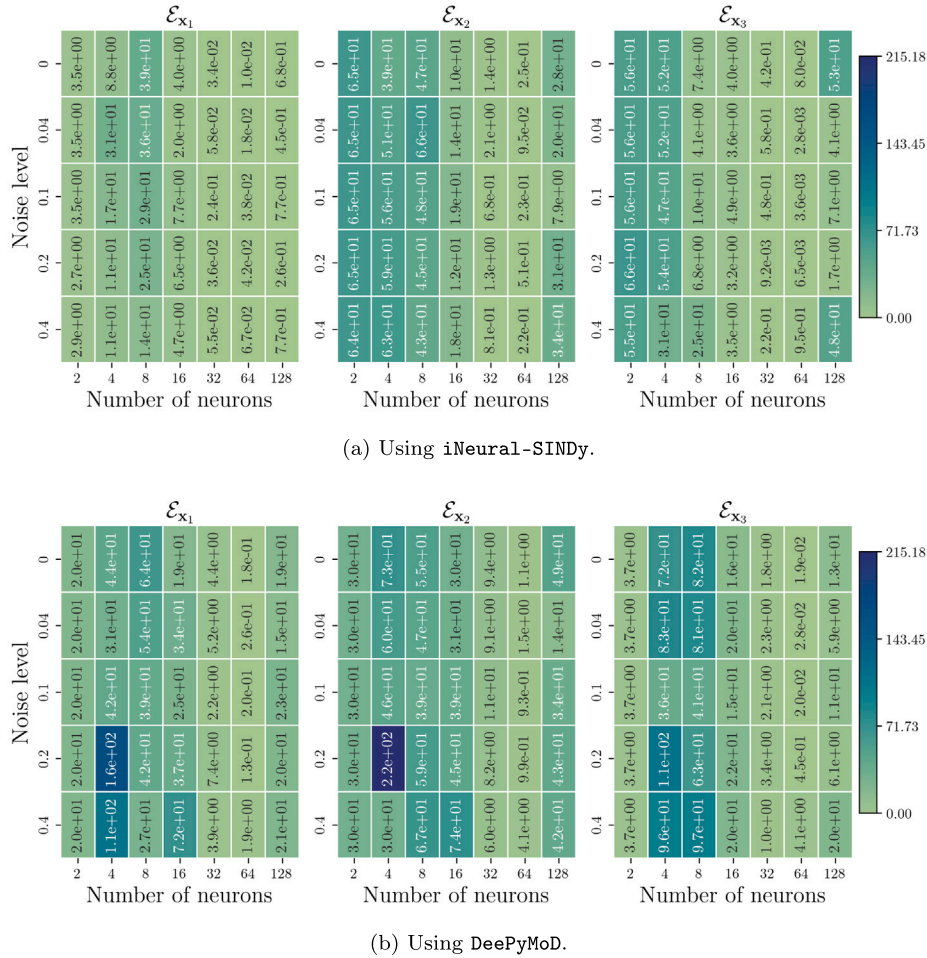


Fig. 18. Lorenz example: A comparison of iNeural-SINDy and DeePyMoD under Scene_B with the scaling factor $\alpha = 1$.

Table A.1
Estimated coefficients for linear damped oscillator.

Learned equations			
Noise level	iNeural-SINDy	DeePyMoD (Both et al., 2021)	RK4-SINDy (Goyal and Benner, 2022)
0.35	$\dot{\mathbf{x}}_1(t) = -0.108\mathbf{x}_1(t) + 1.975\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = -2.025\mathbf{x}_1(t) - 0.090\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -0.128\mathbf{x}_1(t) + 1.978\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = -2.017\mathbf{x}_1(t) - 0.0831\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.057\mathbf{x}_1(t) - 3.300\mathbf{x}_1(t)$ $- 1.126\mathbf{x}_2(t) + 0.191\mathbf{x}_1^2(t)$ $+ 0.104\mathbf{x}_1(t)\mathbf{x}_2(t) - 0.276\mathbf{x}_2^2(t)$
			$\dot{\mathbf{x}}_2(t) = -0.099 - 5.014\mathbf{x}_1(t)$ $- 2.997\mathbf{x}_2(t) - 0.225\mathbf{x}_1(t)\mathbf{x}_2(t)$ $+ 0.127\mathbf{x}_1(t)\mathbf{x}_2(t) - 0.271\mathbf{x}_2^2(t)$
			$\dot{\mathbf{x}}_1(t) = 0.485\mathbf{x}_1(t) - 4.685\mathbf{x}_1(t)$ $- 2.638\mathbf{x}_2(t) - 0.206\mathbf{x}_1^2(t)$ $- 0.169\mathbf{x}_2^2(t)$
0.45	$\dot{\mathbf{x}}_1(t) = -0.108\mathbf{x}_1(t) + 2.031\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = -1.961\mathbf{x}_1(t) - 0.102\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -0.139\mathbf{x}_1(t) + 2.037\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = -1.975\mathbf{x}_1(t) - 0.066\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 0.562 - 6.415\mathbf{x}_1(t)$ $- 4.555\mathbf{x}_2(t) - 0.231\mathbf{x}_1(t)\mathbf{x}_2(t)$ $+ 0.050\mathbf{x}_1(t)\mathbf{x}_2(t) - 0.186\mathbf{x}_2^2(t)$
			$\dot{\mathbf{x}}_1(t) = 0.0976 - 0.114\mathbf{x}_1(t)$ $+ 1.982\mathbf{x}_2(t) - 0.0653\mathbf{x}_1^2(t)$ $+ 0.0648\mathbf{x}_1(t)\mathbf{x}_2(t)$
			$\dot{\mathbf{x}}_2(t) = -2.071\mathbf{x}_1(t) - 0.0843\mathbf{x}_2(t)$ $- 0.075\mathbf{x}_1(t)\mathbf{x}_2$
0.5	$\dot{\mathbf{x}}_1(t) = -0.113\mathbf{x}_1(t) + 1.934\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = -2.088\mathbf{x}_1(t) - 0.094\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -0.160\mathbf{x}_1(t) + 1.982\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = 0.076 - 2.018\mathbf{x}_1(t)$	$\dot{\mathbf{x}}_1(t) = 0.097\mathbf{x}_1(t) - 0.115\mathbf{x}_1(t)$ $1.936\mathbf{x}_2(t) - 0.065\mathbf{x}_1^2(t)$ $- 0.0648\mathbf{x}_1(t)\mathbf{x}_2(t)$
			$\dot{\mathbf{x}}_2(t) = 0.792 - 9.019\mathbf{x}_1(t)$ $- 6.661\mathbf{x}_2(t) - 0.492\mathbf{x}_2^2$
			$\dot{\mathbf{x}}_1(t) = 1.121\mathbf{x}_1(t) - 7.424\mathbf{x}_1(t)$ $- 4.986\mathbf{x}_2 - 0.074\mathbf{x}_1^2(t)$ $- 0.6252\mathbf{x}_2^2(t)$
0.65	$\dot{\mathbf{x}}_1(t) = -0.113\mathbf{x}_1(t) + 1.9641\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = -2.043\mathbf{x}_1(t) - 0.083\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -0.160\mathbf{x}_1(t) + 1.982\mathbf{x}_2(t)$ $\dot{\mathbf{x}}_2(t) = 0.076 - 2.018\mathbf{x}_1(t)$	$\dot{\mathbf{x}}_2(t) = 0.792 - 9.019\mathbf{x}_1(t)$ $- 6.661\mathbf{x}_2(t) - 0.492\mathbf{x}_2^2(t)$

by any additional learning-rate schedules or thresholding steps, both of which can introduce abrupt changes in the training dynamics.

Notes on integration scheme. iNeural-SINDy combines the RK4 scheme with sparse regression. One-step-ahead predictions allow the framework to iteratively validate and refine the discovered governing equations while minimizing error accumulation over multiple steps, which can otherwise degrade the sparsity and accuracy of the model. Moreover, in the training process, the one-step-ahead RK4 prediction forms a part of the loss function. Keeping the prediction horizon short simplifies the optimization, reduces computational cost, and avoids compounding integration errors during backpropagation, which can arise from long-horizon multi-step predictions. It is worth to highlight that, one of the primary reasons iNeural-SINDy is more robust to noise than a purely RK4-based SINDy approach is the built-in denoising mechanism provided by the neural network. Rather than applying integration directly to raw, noisy measurements, iNeural-SINDy first learns a smooth latent representation of the state via a neural network. This latent representation mitigates noise in the data before the SINDy library construction and coefficient fitting take place.

Extensions to partial differential equations. Partial Differential Equations (PDEs) pose unique challenges in sparse system identification, requiring sophisticated sampling strategies for both spatial and temporal dimensions due to their high-dimensional nature. Unlike ODEs, PDEs involve complex spatial dependencies, which demand adaptive and efficient sampling methods to ensure accurate recovery of dynamics. Approaches such as Greedy Sampling (e.g., Q-DEIM (Forootani and Benner, 2024; Forootani et al., 2024b)) have been employed recently to select the most informative spatio-temporal points for PDE training, significantly reducing computational cost while retaining essential system features.

6. Conclusions

In this work, we proposed a methodology, iNeural-SINDy, for discovering governing equations from noisy and scarce data. The framework consists of three main components: (a) learning an implicit representation of the given noisy data using a deep neural network, (b) formulating a sparse regression problem inspired by SINDy (Brunton et al., 2016a) to identify governing equations, and (c) incorporating an integral form of differential equations. By combining these components, iNeural-SINDy enables the accurate identification of governing equations, even under challenging data conditions. Notably, the methodology leverages implicit neural network representations to estimate derivatives via automatic differentiation, thereby avoiding numerical derivative computation, which is prone to error in noisy datasets.

We demonstrated the capability of iNeural-SINDy to effectively handle data collected from multiple trajectories. Furthermore, we conducted extensive comparisons of the proposed methodology with RK4-SINDy (Goyal and Benner, 2022) and DeePyMoD (Both et al., 2021), showing that iNeural-SINDy clearly outperforms both methods in the noisy dataset regime. Additionally, we compared iNeural-SINDy with Weak-SINDy on the Lorenz system, where iNeural-SINDy consistently demonstrated superior performance. The iNeural-SINDy algorithm is highly effective for modeling biological systems such as gene regulatory networks and neuronal dynamics (inspired by the FHN model), as well as single- and multi-degree-of-freedom oscillatory systems, facilitating the robust discovery of governing equations in noisy and nonlinear dynamical settings (Lathourakis and Cicirello, 2024; Izhikevich, 2007).

Simulations shown that by jointly minimizing MSE between network output and noisy measurements, the mismatch between neural-network derivatives and the sparse regression model, and a time-stepping/integral (e.g., RK4-SINDy) consistency loss in a single unified framework,

Table A.2
Estimated coefficients for Cubic damped oscillator.

Estimated system	Noise level	iNeural-SINDy	DeePyMoD (Both et al., 2021)	RK4-SINDy (Goyal and Benner, 2022)
	0.08	$\dot{x}_1(t) = -0.100x_1^3(t) + 1.925x_2^3(t)$ $\dot{x}_2(t) = -2.07x_1^3(t) - 0.103x_2^3(t)$	$\dot{x}_1(t) = -0.121x_1^3(t) + 0.086x_1(t)x_2^2(t) + 2.007x_2^3(t)$ $\dot{x}_2(t) = -2.008x_1^3(t) + 0.066x_1^2(t)x_2(t) - 0.131x_2^3(t)$	$\dot{x}(t) = -0.380x_1(t) - 0.167x_2(t) - 0.051x_1^2(t) + 0.056x_1(t)x_2(t) - 0.255x_1^3(t) + 0.203x_1^2(t)$ $+ 0.062x_1(t)x_2^2(t) + 1.665x_2^3(t)$ $\dot{x}_2(t) = -0.507x_1(t) - 0.288x_2(t) + 0.103x_1(t)x_2(t) - 2.101x_1^3(t) + 0.219x_1^2(t)x_2(t)$ $+ 0.130x_1x_2^2 - 0.355x_2^3(t)$
	0.10	$\dot{x}_1(t) = -0.108x_1^3(t) + 1.972x_2^3(t)$ $\dot{x}_2(t) = -2.02x_1^3(t) - 0.0949x_2^3(t)$	$\dot{x}_1(t) = +0.152x_2(t) - 0.106x_1^3(t) + 1.74x_2^3(t)$ $\dot{x}_2(t) = +0.118x_1(t) - 2.07x_1^3(t) - 0.136x_1(t)x_2^2(t)$ $- 0.113x_2^3(t)$	$\dot{x}(t) = -0.405x_1(t) - 0.130x_2(t) - 0.051x_1^2(t) - 0.370x_1^3(t) + 0.213x_1^2(t)x_2(t) + 1.461x_2^3(t)$ $\dot{x}_2(t) = -0.380x_1(t) - 0.122x_2(t) + 0.061x_1(t)x_2(t) - 2.286x_1^3(t) + 0.204x_1^2(t)x_2(t)$ $- 0.585x_2^3(t) - 0.054$
	0.20	$\dot{x}_1(t) = -0.108x_1^3(t) + 1.973x_2^3(t)$ $\dot{x}_2(t) = -2.02x_1^3(t) - 0.095x_2^3(t)$	$\dot{x}_1(t) = 0.153x_2(t) - 0.106x_1^3(t) + 1.74x_2^3(t)$ $\dot{x}_2(t) = 0.118x_2(t) - 2.207x_1^3(t) - 0.136x_1x_2^2$ $- 0.113x_2^3(t)$	$\dot{x}(t) = -0.405x_1(t) - 0.130x_2(t) - 0.370x_1^3(t) + 0.213x_1^2(t)x_2(t) - 1.461x_2^3(t)$ $\dot{x}_2(t) = -0.380x_1(t) - 0.122x_2(t) + 0.062x_1(t)x_2(t) - 2.286x_1^3(t) + 0.204x_1^2(t)x_2(t) - 0.585x_2^3$
	0.25	$\dot{x}_1(t) = -0.075x_1^3(t) + 2.032x_2^3(t)$ $\dot{x}_2(t) = -2.037x_1^3(t)$ $- 0.137x_1^2(t)x_2(t) - 0.084x_2^3(t)$	$\dot{x}_1(t) = 0.060x_2(t) - 0.090x_1^3(t) - 0.117x_1^2(t)x_2(t)$ $+ 1.98x_2^3(t)$ $\dot{x}_2(t) = -0.068x_1(t) - 0.05x_2(t) - 0.037x_1^2(t)$ $- 2.207x_1^3(t) - 0.068x_2^3(t)$	$\dot{x}(t) = -0.157 - 1.658x_1(t) - 0.093x_2(t) + 1.176x_1(t)x_2(t) + 0.13x_2^2(t) - 1.24x_1^3(t)$ $+ 0.937x_1^2(t)x_2(t) + 0.057x_1(t)x_2^2(t) - 0.49x_2^3(t)$ $\dot{x}_2(t) = -0.08 - 1.962x_1(t) - 0.122x_2(t) + 0.427x_1(t)x_2(t) - 2.93x_1^3(t)$ $+ 0.838x_1^2(t)x_2(t) + 0.18x_1(t)x_2^2(t) - 0.585x_2^3$
	0.30	$\dot{x}_1(t) = -0.075x_1^3(t) + 2.032x_2^3(t)$ $\dot{x}_2(t) = -2.037x_1^3(t) - 0.137x_1^2(t)x_2(t)$ $- 0.084x_2^3(t)$	$\dot{x}(t) = 0.0916x_1(t) - 0.248x_1^3(t) + 0.107x_1(t)x_2^2(t)$ $+ 1.912x_2^3(t)$ $\dot{x}_2(t) = -0.118x_2(t) - 0.108x_1^2(t) - 0.0596x_1(t)x_2(t)$ $- 2.0371x_1^3(t) + 0.265x_1^2(t)x_2(t)$ $- 0.066x_2^3$	$\dot{x}(t) = 0.272 + 0.644x_2(t) - 0.105x_1(t)x_2(t) - 0.425x_1^2(t) - 0.151x_1(t)x_2(t) - 2.754x_2^2(t)$ $+ 0.377x_1^3(t) - 0.764x_1^2(t)x_2(t) - 1.695x_2^3(t)$ $\dot{x}_2(t) = 0.49 - 0.586x_1(t) + 1.194x_2(t) - 0.307x_1^2(t) - 0.447x_1(t)x_2(t) - 0.126x_1^2(t) - 4.132x_1^3(t)$ $+ 0.120x_1^2(t)x_2(t) - 0.85x_1(t)x_2^2(t) - 3.850x_2^3$
	0.35	$\dot{x}_1(t) = -0.060x_1(t) + 0.102x_2(t)$ $+ 0.137x_1(t)x_2(t) - 0.075x_1^3(t)$ $+ 0.175x_1^2(t)x_2(t) + 1.796x_2^3(t)$ $\dot{x}_2(t) = 0.079x_1(t) + 0.0620x_2(t)$ $- 0.063x_1^2(t) - 0.090x_1^2(t)x_2(t)$ $- 1.93x_1^3(t) - 0.214x_1^2(t)x_2(t)$ $- 0.198x_1(t)x_2^2(t) - 0.187x_2^3(t)$	$\dot{x}_1(t) = -0.121x_2(t) - 0.087x_1^2(t) - 0.182x_1(t)x_2(t)$ $- 0.171x_2^2(t) - 0.202x_1^3(t)$ $+ 0.176x_1^2(t)x_2(t) + 0.245x_1(t)x_2^2(t)$ $+ 2.017x_2^3(t) + 0.189$ $\dot{x}_2(t) = -0.107x_1(t) + 0.132x_2^2(t) - 1.905x_1^3(t)$ $+ 0.263x_1^2(t)x_2(t) - 0.176x_2^3(t) + 0.273$	$\dot{x}_1(t) = -0.121x_2(t) - 0.087x_1^2(t) - 0.182x_1(t)x_2(t) - 0.171x_2^2(t) - 0.202x_1^3(t) + 0.176x_1^2(t)x_2(t)$ $+ 0.245x_1(t)x_2^2(t) + 2.017x_2^3(t) + 0.112$ $\dot{x}_2(t) = -0.107x_1(t) + 0.132x_2^2(t) - 1.905x_1^3(t) + 0.263x_1^2(t)x_2(t) - 0.176x_2^3(t) - 0.0813$

our method becomes substantially more robust to high levels of noise and irregular data sampling.

The iNeural-SINDy methodology can enhance control systems by providing robust, sparse models for use in frameworks like Model Predictive Control (MPC), particularly in scenarios with noisy and limited data. While the basic SINDy algorithm, as demonstrated in Kaiser et al. (2018), effectively identifies sparse models for MPC, iNeural-SINDy improves upon this by leveraging neural networks and integral constraints to handle noise and irregular data. This makes it ideal for adaptive control, feedback linearization, and reduced-order modeling, ensuring computational efficiency and accuracy in real-time control applications.

In future work, we aim to extend the proposed framework to identify parametric and control-driven dynamical systems. Additionally, we plan to incorporate the ensemble techniques discussed in Fasel et al. (2022) to further enhance the quality of the discovered governing equations.

CRediT authorship contribution statement

Ali Forootani: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Pawan Goyal:** Writing – review & editing, Validation, Investigation, Formal analysis. **Peter Benner:** Visualization, Validation, Supervision, Project administration.

Code and data availability statement.

iNeural-SINDy is implemented using the PyTorch neural network framework, leveraging its robust capabilities for building and

training deep learning models. The codebase, including detailed examples and documentation, is publicly accessible in the following GitHub repository² or Zenodo³. This repository provides comprehensive resources for replicating and extending the methods described in the associated research paper.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

A.1. Tables

In this appendix, we present a comparison of learned governing equations using various methods.

Data availability

No data was used for the research described in the article.

² https://github.com/Ali-Forootani/iNeural_SINDy_paper/tree/main

³ <https://zenodo.org/records/13906751>, DOI: 10.5281/zenodo.13906751

Table A.3
Estimated coefficients for Fitz-Hugh Nagumo.

Estimated system			
Noise level	iNeural-SINDy	DeePyMoD (Both et al., 2021)	RK4-SINDy (Goyal and Benner, 2022)
0.00	$\dot{\mathbf{x}}_1(t) = 0.989\mathbf{x}_1(t) - 0.993\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.992\mathbf{x}_1(t) - 0.994\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.986\mathbf{x}_1(t) - 0.996\mathbf{x}_2(t)$
	$- 0.329\mathbf{x}_1^3(t) + 0.100$	$- 0.330\mathbf{x}_1^3(t) + 0.100$	$- 0.328\mathbf{x}_1^3(t) + 0.0993$
	$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.099\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.100\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.099\mathbf{x}_2(t)$
0.04	$\dot{\mathbf{x}}_1(t) = 0.997\mathbf{x}_1(t) - 0.999\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.967\mathbf{x}_1(t) - 0.973\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 1.000\mathbf{x}_1(t) - 1.010\mathbf{x}_2(t)$
	$- 0.332\mathbf{x}_1(t)^3 + 0.101$	$- 0.321\mathbf{x}_1(t)^3 + 0.098$	$- 0.335\mathbf{x}_1(t)^3 + 0.1$
	$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.102\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.100\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 0.101\mathbf{x}_1(t) - 0.109\mathbf{x}_2(t)$
0.08	$\dot{\mathbf{x}}_1(t) = 0.978\mathbf{x}_1(t) - 0.985\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.908\mathbf{x}_1(t) - 0.941\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.336\mathbf{x}_1(t) - 0.081\mathbf{x}_2^2(t)$
	$- 0.324\mathbf{x}_1(t)^3 + 0.097$	$- 0.297\mathbf{x}_1(t)^3 + 0.096$	$- 0.076\mathbf{x}_1^3 - 0.117\mathbf{x}_1^2(t)\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.112\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 0.102\mathbf{x}_1(t) - 0.106\mathbf{x}_2(t)$	$- 0.139\mathbf{x}_1(t)\mathbf{x}_2^2(t) - 0.113\mathbf{x}_2^3(t)$
0.1			$\dot{\mathbf{x}}_2(t) = 0.098\mathbf{x}_1(t) - 0.062\mathbf{x}_2(t)^2$
			$- 0.220\mathbf{x}_2^3$
	$\dot{\mathbf{x}}_1(t) = 0.904\mathbf{x}_1(t) - 0.9737\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.814\mathbf{x}_1(t) - 1.069\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 1.043\mathbf{x}_1(t) - 0.877\mathbf{x}_2(t)$
0.12	$- 0.301\mathbf{x}_1^3(t) + 0.1017$	$- 0.070\mathbf{x}_1^2(t) - 0.257\mathbf{x}_2^2(t)$	$+ 0.278\mathbf{x}_2^2(t) - 0.348\mathbf{x}_1^3(t)$
	$\dot{\mathbf{x}}_2(t) = 0.101\mathbf{x}_1(t) - 0.115\mathbf{x}_2(t)$	$- 0.264\mathbf{x}_1^3(t) + 0.120\mathbf{x}_1^2(t)\mathbf{x}_2(t)$	$- 0.0526\mathbf{x}_1^2(t)\mathbf{x}_2(t) + 0.162\mathbf{x}_2^3(t)$
		$+ 0.162\mathbf{x}_2^3(t) + 0.281$	$\dot{\mathbf{x}}_2(t) = 0.362\mathbf{x}_1(t) - 0.118\mathbf{x}_2(t)^2$
0.12		$\dot{\mathbf{x}}_2(t) = 0.100\mathbf{x}_1(t) - 0.100\mathbf{x}_2(t)$	$- 0.085\mathbf{x}_1^3(t) - 0.116\mathbf{x}_1^2(t)\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_1(t) = 0.855\mathbf{x}_1(t) - 0.995\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 0.655\mathbf{x}_1(t) - 0.983\mathbf{x}_2(t)$	$- 0.145\mathbf{x}_1\mathbf{x}_2^2(t) - 0.166\mathbf{x}_2^3(t)$
	$- 0.28\mathbf{x}_1^3(t) + 0.075\mathbf{x}_1^2(t)\mathbf{x}_2(t) + 0.090$	$- 0.068\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 1.123\mathbf{x}_1(t) - 0.903\mathbf{x}_2(t)$
0.12	$\dot{\mathbf{x}}_2(t) = 0.105\mathbf{x}_1(t) - 0.106\mathbf{x}_2(t)$	$- 0.218\mathbf{x}_1^3 + 0.088\mathbf{x}_1^2(t)\mathbf{x}_2(t)$	$+ 0.076\mathbf{x}_2^2 - 0.371\mathbf{x}_1^3$
		$+ 0.218\mathbf{x}_1(t)\mathbf{x}_2^2(t) + 0.102$	$- 0.078\mathbf{x}_1^2(t)\mathbf{x}_2(t)$
		$\dot{\mathbf{x}}_2(t) = 0.101\mathbf{x}_1(t) - 0.116\mathbf{x}_2(t)$	$- 0.079\mathbf{x}_1(t)\mathbf{x}_2^2(t) + 0.088$
0.12			$\dot{\mathbf{x}}_2(t) = 0.423\mathbf{x}_1(t) + 0.106\mathbf{x}_1^3(t)$
			$- 0.146\mathbf{x}_1^2(t)\mathbf{x}_2(t)$
			$- 0.166\mathbf{x}_1(t)\mathbf{x}_2^2(t) - 0.147\mathbf{x}_2^3(t)$

Table A.4
Estimated coefficients for Lorenz.

Estimated system			
Noise level	iNeural-SINDy	DeePyMoD (Both et al., 2021)	RK4-SINDy (Goyal and Benner, 2022)
0.00	$\dot{\mathbf{x}}_1(t) = -9.989\mathbf{x}_1(t) + 9.991\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.992\mathbf{x}_1(t) + 9.993\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.995\mathbf{x}_1(t) + 10.003\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 28.022\mathbf{x}_1(t) - 1.004\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 28.026\mathbf{x}_1(t) - 1.000\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 28.032\mathbf{x}_1(t) - 1.000\mathbf{x}_2(t)$
	$+ 10.006\mathbf{x}_1(t)\mathbf{x}_3(t)$	$+ 10.000\mathbf{x}_1(t)\mathbf{x}_3(t)$	$+ 10.014\mathbf{x}_1(t)\mathbf{x}_3(t)$
0.10	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 10.013\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 10.000\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.665\mathbf{x}_3(t) + 10.020\mathbf{x}_1(t)\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_1(t) = -9.982\mathbf{x}_1(t) + 9.985\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.997\mathbf{x}_1(t) + 9.994\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -10.004\mathbf{x}_1(t) + 10.008\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 28.001\mathbf{x}_1(t) - 1.001\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 28.028\mathbf{x}_1(t) - 1.005\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 28.060\mathbf{x}_1(t) - 0.997\mathbf{x}_2(t)$
0.20	$- 10.005\mathbf{x}_1(t)\mathbf{x}_3(t)$	$- 10.008\mathbf{x}_1(t)\mathbf{x}_3(t)$	$- 10.025\mathbf{x}_1(t)\mathbf{x}_3(t)$
	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 10.008\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 9.999\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.664\mathbf{x}_3(t) + 10.030\mathbf{x}_1(t)\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_1(t) = -10.016\mathbf{x}_1(t) + 10.008\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -10.034\mathbf{x}_1(t) + 10.032\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -10.076\mathbf{x}_1(t) + 10.068\mathbf{x}_2(t)$
0.40	$\dot{\mathbf{x}}_2(t) = 27.980\mathbf{x}_1(t) - 0.991\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 27.968\mathbf{x}_1(t) - 0.998\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 27.846\mathbf{x}_1(t) - 0.928\mathbf{x}_2(t)$
	$- 9.998\mathbf{x}_1(t)\mathbf{x}_3(t)$	$- 9.986\mathbf{x}_1(t)\mathbf{x}_3(t)$	$- 9.967\mathbf{x}_1(t)\mathbf{x}_3(t)$
	$\dot{\mathbf{x}}_3(t) = -2.670\mathbf{x}_3(t) + 10.010\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.669\mathbf{x}_3(t) + 9.993\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.674\mathbf{x}_3(t) + 9.995\mathbf{x}_1(t)\mathbf{x}_2(t)$
0.40	$\dot{\mathbf{x}}_1(t) = -10.122\mathbf{x}_1(t) + 10.080\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.887\mathbf{x}_1(t) + 9.902\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -10.976\mathbf{x}_1(t) + 10.272\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 27.714\mathbf{x}_1(t) - 0.920\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_2(t) = 26.627\mathbf{x}_1(t) - 9.596\mathbf{x}_1(t)\mathbf{x}_3(t)$	$+ 0.225\mathbf{x}_1(t)\mathbf{x}_3(t)$
	$- 9.938\mathbf{x}_1(t)\mathbf{x}_3(t)$	$- 0.332\mathbf{x}_2(t)\mathbf{x}_3(t)$	$\dot{\mathbf{x}}_2(t) = 26.144\mathbf{x}_1(t) - 9.490\mathbf{x}_1(t)\mathbf{x}_3(t)$
0.40	$\dot{\mathbf{x}}_3(t) = -2.673\mathbf{x}_3(t) + 9.933\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.657\mathbf{x}_3(t) + 0.035\mathbf{x}_1(t)\mathbf{x}_3(t)$	$- 0.270\mathbf{x}_2(t)\mathbf{x}_3(t)$
		$+ 9.964\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.683\mathbf{x}_3(t) + 10.030\mathbf{x}_1(t)\mathbf{x}_2(t)$

Table A.5
Comparison of iNeural-SINDy with WEAK-SINDy (Messenger and Bortz, 2021).

Estimated System		
Noise level	iNeural-SINDy	Weak-SINDy
0.00	$\dot{\mathbf{x}}_1(t) = -9.990\mathbf{x}_1(t) + 9.990\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = 10.000\mathbf{x}_1(t) - 10.000\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 28.021\mathbf{x}_1(t) - 1.003\mathbf{x}_2(t) - 1.006\mathbf{x}_1(t)\mathbf{x}_3(t)$	$\dot{\mathbf{x}}_2(t) = 28.026\mathbf{x}_1(t) - 1.000\mathbf{x}_2(t) - 1.000\mathbf{x}_1(t)\mathbf{x}_3(t)$
	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 1.001\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.670\mathbf{x}_3(t) + 1.000\mathbf{x}_1(t)\mathbf{x}_2(t)$
0.02	$\dot{\mathbf{x}}_1(t) = -9.989\mathbf{x}_1(t) + 9.991\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.908\mathbf{x}_1(t) + 9.926\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 28.013\mathbf{x}_1(t) - 1.001\mathbf{x}_2(t) - 1.000\mathbf{x}_1(t)\mathbf{x}_3(t)$	$\dot{\mathbf{x}}_2(t) = 27.857\mathbf{x}_1(t) - 0.991\mathbf{x}_2(t) - 0.998\mathbf{x}_1(t)\mathbf{x}_3(t)$
	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 1.001\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.658\mathbf{x}_3(t) + 1.002\mathbf{x}_1(t)\mathbf{x}_2(t)$
0.08	$\dot{\mathbf{x}}_1(t) = -9.989\mathbf{x}_1(t) + 9.991\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.798\mathbf{x}_1(t) + 9.778\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 27.991\mathbf{x}_1(t) - 0.996\mathbf{x}_2(t) - 0.999\mathbf{x}_1(t)\mathbf{x}_3(t)$	$\dot{\mathbf{x}}_2(t) = 26.699\mathbf{x}_1(t) - 0.790\mathbf{x}_2(t) - 0.969\mathbf{x}_1(t)\mathbf{x}_3(t)$
	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 1.001\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.640\mathbf{x}_3(t) + 1.010\mathbf{x}_1(t)\mathbf{x}_2(t)$
0.10	$\dot{\mathbf{x}}_1(t) = -9.988\mathbf{x}_1(t) + 9.991\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_1(t) = -9.751\mathbf{x}_1(t) + 9.734\mathbf{x}_2(t)$
	$\dot{\mathbf{x}}_2(t) = 27.982\mathbf{x}_1(t) - 0.993\mathbf{x}_2(t) - 0.994\mathbf{x}_1(t)\mathbf{x}_3(t)$	$\dot{\mathbf{x}}_2(t) = 26.337\mathbf{x}_1(t) - 0.716\mathbf{x}_1(t)\mathbf{x}_3(t) - 0.961\mathbf{x}_2(t)\mathbf{x}_3(t)$
	$\dot{\mathbf{x}}_3(t) = -2.666\mathbf{x}_3(t) + 1.019\mathbf{x}_1(t)\mathbf{x}_2(t)$	$\dot{\mathbf{x}}_3(t) = -2.630\mathbf{x}_3(t) + 1.010\mathbf{x}_1(t)\mathbf{x}_2(t)$

References

Arlot, Sylvain, Celisse, Alain, 2010. A survey of cross-validation procedures for model selection. *Stat. Surv.* 4 (none), 40–79.

Beck, Amir, Eldar, Yonina C., 2013. Sparsity constrained nonlinear optimization: Optimality conditions and algorithms. *SIAM J. Optim.* 23 (3), 1480–1509.

Beetham, Sarah, Capece, Jesse, 2020. Formulating turbulence closures using sparse regression with embedded form invariance. *Phys. Rev. Fluids* 5 (8), 084611.

Bergstra, James, Bengio, Yoshua, 2012. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.* 13 (2).

Boninsegna, Lorenzo, Nüske, Feliks, Clementi, Cecilia, 2018. Sparse learning of stochastic dynamical equations. *J. Chem. Phys.* 148 (24), 241723.

Both, Gert-Jan, Choudhury, Subham, Sens, Pierre, Kusters, Remy, 2021. DeepMoD: Deep learning for model discovery in noisy data. *J. Comput. Phys.* 428, 109985.

Brunton, Steven L., Proctor, Joshua L., Kutz, J. Nathan, 2016a. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proc. Natl. Acad. Sci. USA* 113 (15), 3932–3937.

Brunton, Steven L., Proctor, Joshua L., Kutz, J. Nathan, 2016b. Sparse identification of nonlinear dynamics with control (SINDy). *IFAC-Pap.* 49 (18), 710–715.

Brunton, Steven L., Tu, Jonathan H., Bright, Ido, Kutz, J. Nathan, 2014. Compressive sensing and low-rank libraries for classification of bifurcation regimes in nonlinear dynamical systems. *SIAM J. Appl. Dyn. Syst.* 13 (4), 1716–1732.

Buckwar, Evelyn, Winkler, Renate, 2006. Multistep methods for SDEs and their application to problems with small noise. *SIAM J. Numer. Anal.* 44 (2), 779–803.

Butcher, John Charles, 2016. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons.

Callahan, Jared L., Loiseau, J-C, Rigas, Georgios, Brunton, Steven L., 2021. Nonlinear stochastic modelling with langevin regression. *Proc. Roy. Soc. A* 477 (2250), 20210092.

Chen, Ricky TQ, Rubanova, Yulia, Bettencourt, Jesse, Duvenaud, David K., 2018. Neural ordinary differential equations. *Adv. Neural Inf. Process. Sys.* 31.

Crutchfield, James P., McNamara, Bruce S., 1987. Equations of motion from a data series. *Complex Sys.* 1, 417–452.

Dam, Magnus, Bröns, Morten, Juul, Rasmussen, Jens, Naulin, Volker, Hesthaven, Jan S., 2017. Sparse identification of a predator-prey system from simulation data of a convection model. *Phys. Plasmas* 24 (2).

Fasel, Urban, Kutz, J. Nathan, Brunton, Bingni W., Brunton, Steven L., 2022. Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control. *Proc. Roy. Soc. A* 478 (2260), 20210904.

Feurer, Matthias, Hutter, Frank, 2019. Hyperparameter optimization. *Autom. Mach. Learn.: Methods, Syst. Chall.* 3–33.

Forootani, Ali, Benner, Peter, 2024. GN-SINDy: Greedy sampling neural network in sparse identification of nonlinear partial differential equations. *arXiv preprint arXiv:2405.08613*.

Forootani, Ali, Iervolino, Raffaele, Tipaldi, Massimo, 2019. Applying unweighted least-squares based techniques to stochastic dynamic programming: theory and application. *IET Control Theory Appl.* 13 (15), 2387–2398.

Forootani, Ali, Iervolino, Raffaele, Tipaldi, Massimo, Baccari, Silvio, 2024a. A kernel-based approximate dynamic programming approach: Theory and application. *Automatica* 162, 111517.

Forootani, Ali, Kapadia, Harshit, Chellappa, Sridhar, Goyal, Pawan, Benner, Peter, 2024b. GS-PINN: Greedy sampling for parameter estimation in partial differential equations. *arXiv preprint arXiv:2405.08537*.

Forootani, Ali, Tipaldi, Massimo, Iervolino, Raffaele, Dey, Subhrakanti, 2021. Enhanced exploration least-squares methods for optimal stopping problems. *IEEE Control. Syst. Lett.* 6, 271–276.

Forootani, Ali, Zarch, Majid Ghaniee, Tipaldi, Massimo, Iervolino, Raffaele, 2023. A stochastic dynamic programming approach for the machine replacement problem. *Eng. Appl. Artif. Intell.* 118, 105638.

Goyal, Pawan, Benner, Peter, 2022. Discovery of nonlinear dynamical systems using a runge-kutta inspired dictionary-based sparse regression approach. *Proc. Roy. Soc. A* 478 (2262), 20210883.

Goyal, Pawan, Benner, Peter, 2023. Neural ordinary differential equations with irregular and noisy data. *Roy. Soc. Open Sci.* 10 (7), 221475.

Hastie, Trevor, Tibshirani, Robert, Friedman, Jerome H., 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, second ed. In: Springer Series in Statistics, Springer.

Izhikevich, Eugene M., 2007. *Dynamical Systems in Neuroscience*. MIT Press.

Kaheman, Kadierdan, Brunton, Steven L., Kutz, J. Nathan, 2022. Automatic differentiation to simultaneously identify nonlinear dynamics and extract noise probability distributions from data. *Mach. Learn.: Sci. Tech.* 3 (1), 015031.

Kaheman, Kadierdan, Kutz, J. Nathan, Brunton, Steven L., 2020. SINDy-PI: a robust algorithm for parallel implicit sparse identification of nonlinear dynamics. *Proc. Roy. Soc. Edinburgh Sect. A* 476 (2242), 20200279.

Kaiser, Eurika, Kutz, J. Nathan, Brunton, Steven L., 2018. Sparse identification of nonlinear dynamics for model predictive control in the low-data limit. *Proc. Roy. Soc. Edinburgh Sect. A* 474 (2219), 20180335.

Kang, Sung Ha, Liao, Wenjing, Liu, Yingjie, 2021. Ident: Identifying differential equations with numerical time evolution. *J. Sci. Comput.* 87, 1–27.

Kaptanoglu, Alan A., Callahan, Jared L., Aravkin, Aleksandr, Hansen, Christopher J., Brunton, Steven L., 2021. Promoting global stability in data-driven models of quadratic nonlinear dynamics. *Phys. Rev. Fluids* 6 (9), 094401.

Keller, Rachael T., Du, Qiang, 2021. Discovery of dynamics using linear multistep methods. *SIAM J. Numer. Anal.* 59 (1), 429–455.

Kingma, Diederik P., Ba, Jimmy, 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Kumari, S. Narendra, Kannan, Parthasarathy, 1990. Identification and control of dynamical systems using neural networks. *IEEE Trans. Neural Netw.* 1 (1), 4–27.

Kutz, J. Nathan, Brunton, Steven L., Brunton, Bingni W., Proctor, Joshua L., 2016. *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*. SIAM.

Kuznetsov, Nikolay, Reitmann, Volker, 2020. *Attractor Dimension Estimates for Dynamical Systems: Theory and Computation*. Springer.

Lathourakis, Christos, Cicirello, Alice, 2024. Physics enhanced sparse identification of dynamical systems with discontinuous nonlinearities. *Nonlinear Dynam.* 1–28.

Ljung, Lennart, 1999. *System Identification: Theory for the User*. Prentice Hall, NJ.

Loiseau, Jean Christophe, Brunton, Steven L., 2018. Constrained sparse Galerkin regression. *J. Fluid Mech.* 838, 42–67.

Machado, Gabriel F., Jones, Morgan, 2024. Sparse identification of nonlinear dynamics with side information (SINDy-SI). In: 2024 American Control Conference. ACC, IEEE, pp. 2879–2884.

Mackey, Alan, Schaeffer, Hayden, Osher, Stanley, 2014. On the compressive spectral method. *Multiscale Model. Simul.* 12 (4), 1800–1827.

Maddu, Suryanarayana, Cheeseman, Bevan L., Müller, Christian L., Sbalzarini, Ivo F., 2021. Learning physically consistent differential equation models from data using group sparsity. *Phys. Rev. E* 103 (4), 042310.

Messenger, Daniel A., Bortz, David M., 2021. Weak SINDy: Galerkin-based data-driven model selection. *Multiscale Model. Simul.* 19 (3), 1474–1497.

Peterson, Luisa, Forootani, Ali, Sanchez Medina, Edgar Ivan, Gosea, Ion Victor, Sundmacher, Kai, Benner, Peter, 2024. Towards digital twins for power-to-X: Comparing surrogate models for a catalytic CO2 methanation reactor. <http://dx.doi.org/10.36227/Technrxiv.172263007.76668955/V1>.

- Rackauckas, Christopher, Ma, Yingbo, Martensen, Julius, Warner, Collin, Zubov, Kirill, Supekar, Rohit, Skinner, Dominic, Ramadhan, Ali, Edelman, Alan, 2020. Universal differential equations for scientific machine learning. arXiv preprint arXiv:2001.04385.
- Reinbold, Patrick A.K., Gurevich, Daniel R., Grigoriev, Roman O., 2020. Using noisy or incomplete data to discover models of spatiotemporal dynamics. *Phys. Rev. E* 101 (1), 010203.
- Rudy, Samuel H, Brunton, Steven L, Proctor, Joshua L, Kutz, J Nathan, 2017. Data-driven discovery of partial differential equations. *Sci. Adv.* 3 (4), e1602614.
- Schaeffer, Hayden, Caflisch, Russel, Hauck, Cory D, Osher, Stanley, 2013. Sparse dynamics for partial differential equations. *Proc. Natl. Acad. Sci. USA* 110 (17), 6634–6639.
- Schaeffer, Hayden, McCalla, Scott G., 2017. Sparse model selection via integral terms. *Phys. Rev. E* 96 (2), 023302.
- Schmidt, Michael, Lipson, Hod, 2009. Distilling free-form natural laws from experimental data. *Science* 324 (5923), 81–85.
- Schmidt, Michael D, Vallabhajosyula, Ravishankar R, Jenkins, Jerry W, Hood, Jonathan E, Soni, Abhishek S, Wikswo, John P, Lipson, Hod, 2011. Automated refinement and inference of analytical models for metabolic networks. *Phys. Biol.* 8 (5), 055011.
- Sitzmann, Vincent, Martel, Julien, Bergman, Alexander, Lindell, David, Wetzstein, Gordon, 2020. Implicit neural representations with periodic activation functions. *Adv. Neural Inf. Process. Sys.* 33, 7462–7473.
- Sorokina, Mariia, Sygletos, Stylianos, Turitsyn, Sergei, 2016. Sparse identification for nonlinear optical communication systems: SINO method. *Opt. Express* 24 (26), 30433–30443.
- Sprott, Julien Clinton, Sprott, Julien C., 2003. *Chaos and Time-Series Analysis*. 69, Oxford University Press, Oxford.
- Strogatz, Steven H., 2018. *Nonlinear Dynamics and Chaos: with Applications to Physics, Biology, Chemistry, and Engineering*. CRC Press.
- Suykens, Johan A.K., Vandewalle, Joos P.L., de Moor, Bart L., 1996. *Artificial Neural Networks for Modelling and Control of Non-Linear Systems*. Springer.
- Tangirala, A.K., 2018. *Principles of System Identification Theory and Practice*. CRC Press.
- Thaler, Stephan, Paehler, Ludger, Adams, Nikolaus A., 2019. Sparse identification of truncation errors. *J. Comput. Phys.* 397, 108851.
- Tibshirani, Robert, 1996. Regression shrinkage and selection via the lasso. *J. Roy. Stat. Soc.: Ser. B (Methodological)* 58 (1), 267–288.
- Tod, Georges, Both, Gert-Jan, Kusters, Remy, 2021. Discovering PDEs from multiple experiments. arXiv preprint arXiv:2109.11939.
- Van Overschee, Peter, de Moor, Bart, 1996. *Subspace Identification of Linear Systems: Theory, Implementation, Applications*. Kluwer Academic Publishers.
- Wang, Wen-Xu, Yang, Rui, Lai, Ying-Cheng, Kovanis, Vassilios, Grebogi, Celso, 2011. Predicting catastrophes in nonlinear dynamical systems by compressive sensing. *Phys. Rev. Lett.* 106 (15), 154101.
- Yang, Zhuoran, Wang, Zhaoran, Liu, Han, Eldar, Yonina, Zhang, Tong, 2016. Sparse nonlinear regression: Parameter estimation under nonconvexity. In: *Intern. Conf. on Mach. Learn.*. PMLR, pp. 2472–2481.
- Zanna, Laure, Bolton, Thomas, 2020. Data-driven equation discovery of ocean mesoscale closures. *Geophys. Res. Lett.* 47 (17), e2020GL088376.
- Zhang, Sheng, Lin, Guang, 2018. Robust data-driven discovery of governing physical laws with error bars. *Proc. Roy. Soc. A: Math. Phys. Eng. Sci.* 474 (2217), 20180305.
- Zhang, Linan, Schaeffer, Hayden, 2019. On the convergence of the SINDy algorithm. *Multiscale Model. Simul.* 17 (3), 948–972.
- Zolman, Nicholas, Fasel, Urban, Kutz, J Nathan, Brunton, Steven L, 2024. SINDy-RL: Interpretable and efficient model-based reinforcement learning. arXiv preprint arXiv:2403.09110.